WESTERN MICHIGAN UNIVERSITY

Collegue of Engineering and Applied Sciences

Course Name: Lab. Intro. To Microprocessors

Course Number: ECE 2510

Instructors:
Christian Blanchard
Pravana Avuthu

Lab #2

Date: February 3rd 2026

Student:

De Leon Santos, Sebastian Manuel – WIN: 577866168

Table of Contents

Task 1: 3

Task 2: 6

The C Code: 6

Task 2: Assembly hex operations 15

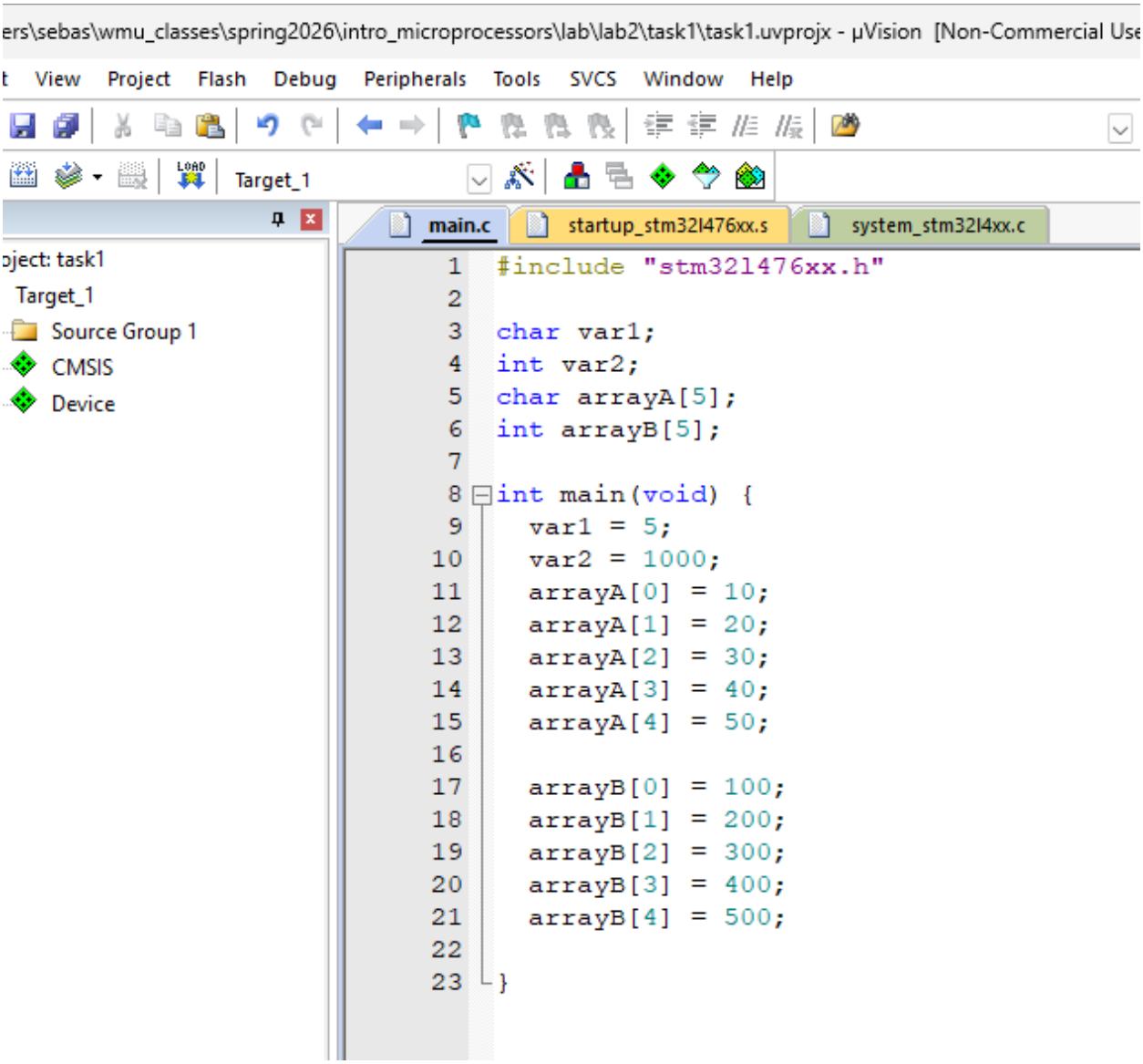
Task 3: 19

Task 5 24

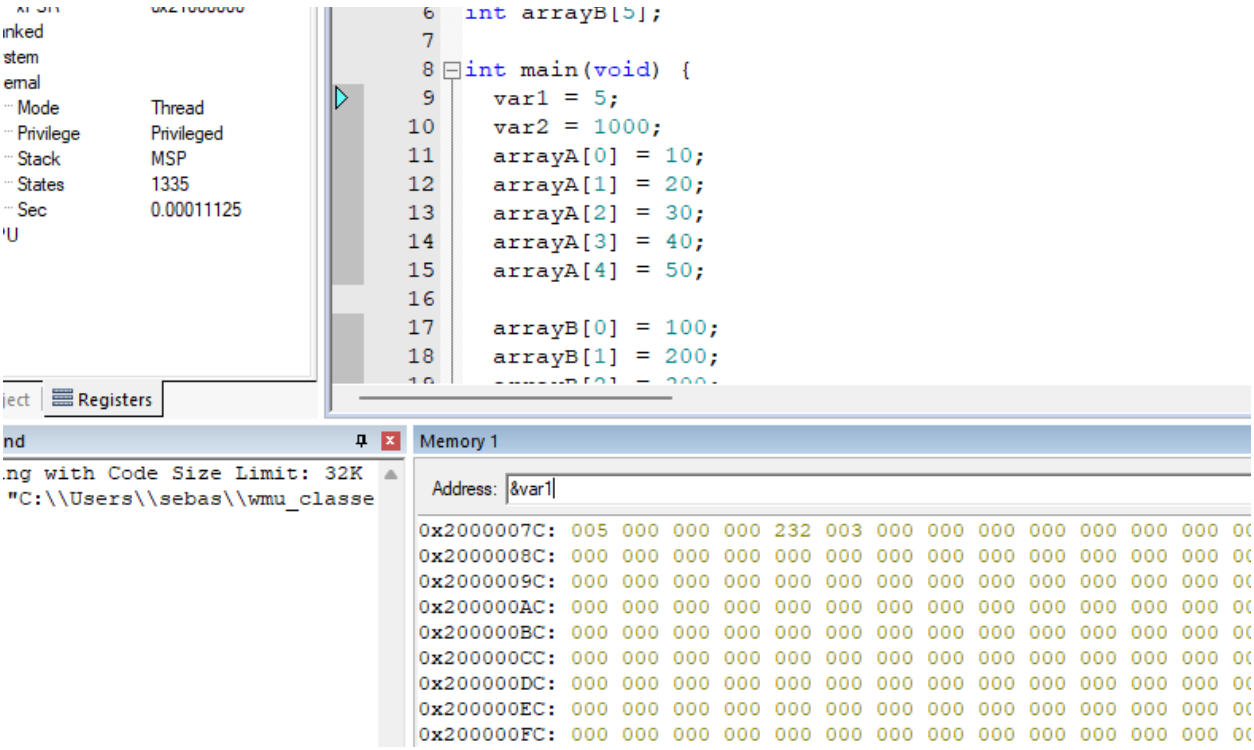
Conclusion 30

Task 1:

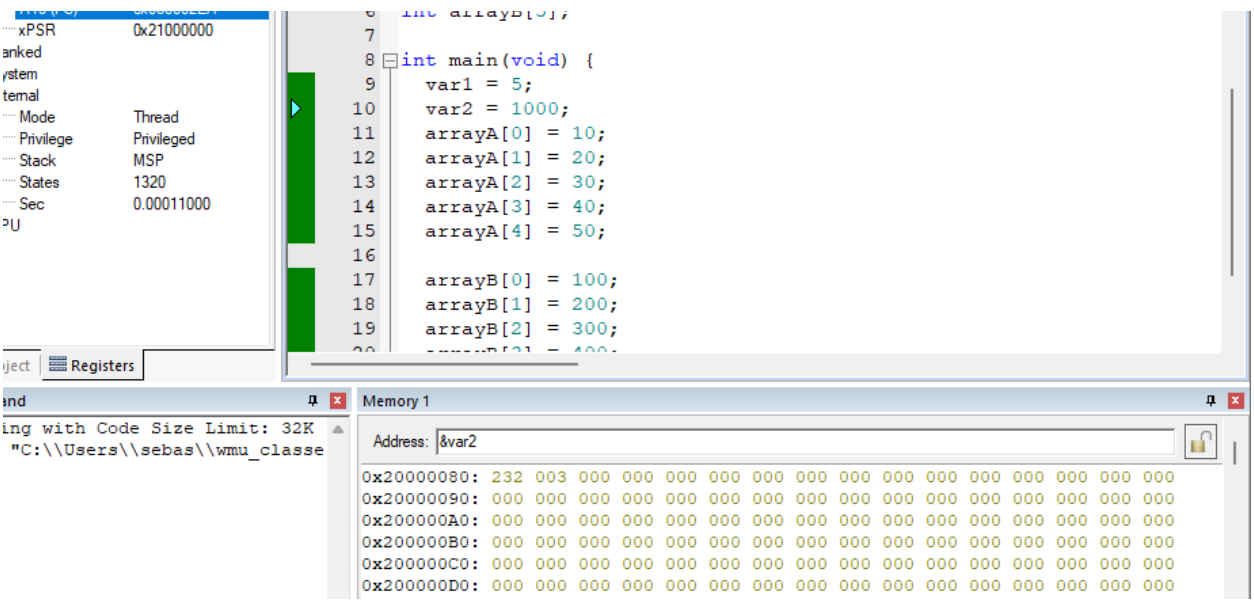
Code:


ers\sebas\wmu_classes\spring2026\intro_microprocessors\lab\lab2\task1\task1.uvprojx - µVision [Non-Commercial Use
View Project Flash Debug Peripherals Tools SVCS Window Help
Target_1
main.c
startup_stm32l476xx.s
system_stm32l4xx.c
oject: task1
Target_1
Source Group 1
CMSIS
Device
#include "stm321476xx.h"
char var1;
int var2;
char arrayA[5];
int arrayB[5];
int main(void) {
var1 = 5;
var2 = 1000;
arrayA[0] = 10;
arrayA[1] = 20;
arrayA[2] = 30;
arrayA[3] = 40;
arrayA[4] = 50;
arrayB[0] = 100;
arrayB[1] = 200;
arrayB[2] = 300;
arrayB[3] = 400;
arrayB[4] = 500;
}

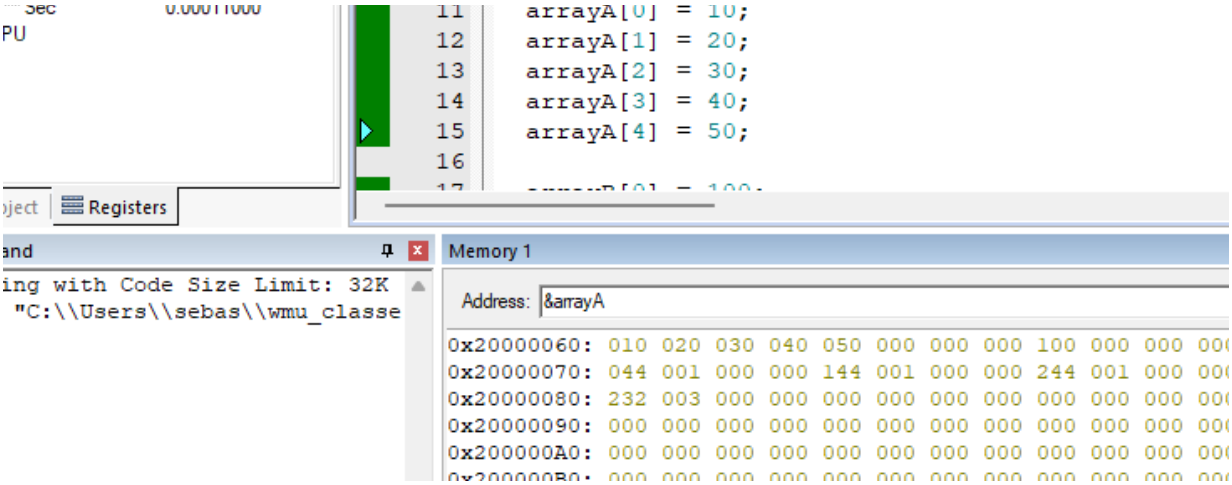
Var1:


Mode Thread
Privilege Privileged
Stack MSP
States 1335
Sec 0.00011125
Registers
int arrayB[5];
int main(void) {
var1 = 5;
var2 = 1000;
arrayA[0] = 10;
arrayA[1] = 20;
arrayA[2] = 30;
arrayA[3] = 40;
arrayA[4] = 50;
arrayB[0] = 100;
arrayB[1] = 200;
Memory 1
Address: &var1
0x2000007C: 005 000 000 000 232 003 000 000 000 000 000 000 000
0x2000008C: 000 000 000 000 000 000 000 000 000 000 000 000 000
0x2000009C: 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000AC: 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000BC: 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000CC: 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000DC: 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000EC: 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000FC: 000 000 000 000 000 000 000 000 000 000 000 000 000

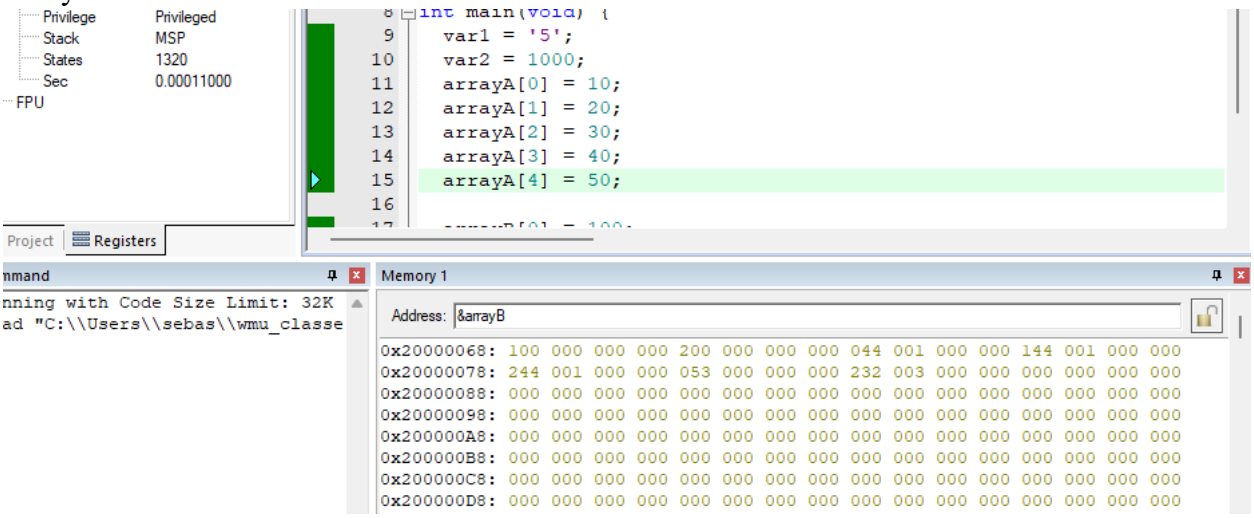
Var2:


xPSR 0x21000000
Mode Thread
Privilege Privileged
Stack MSP
States 1320
Sec 0.00011000
Registers
int main(void) {
var1 = 5;
var2 = 1000;
arrayA[0] = 10;
arrayA[1] = 20;
arrayA[2] = 30;
arrayA[3] = 40;
arrayA[4] = 50;
arrayB[0] = 100;
arrayB[1] = 200;
arrayB[2] = 300;
Memory 1
Address: &var2
0x20000080: 232 003 000 000 000 000 000 000 000 000 000 000 000 000 000 000
0x20000090: 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000A0: 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000B0: 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000C0: 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000
0x200000D0: 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000

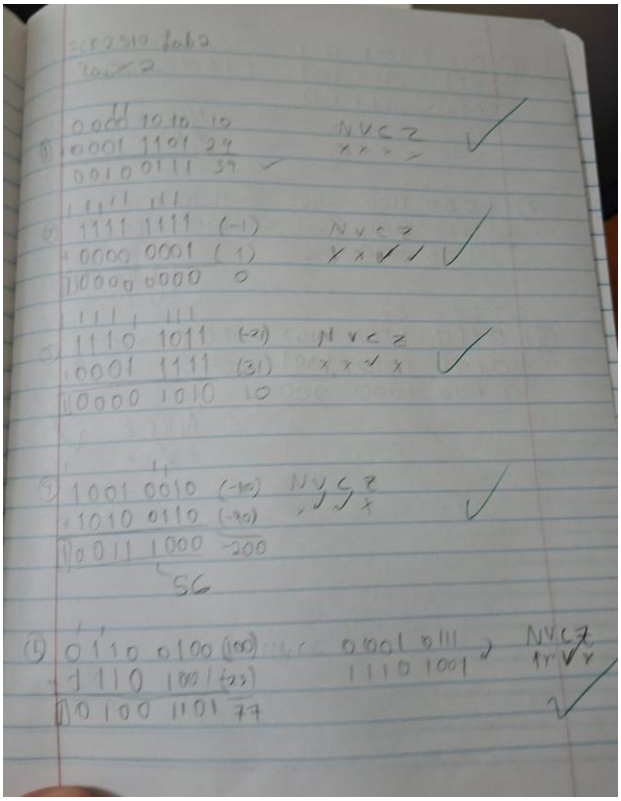
ArrayA

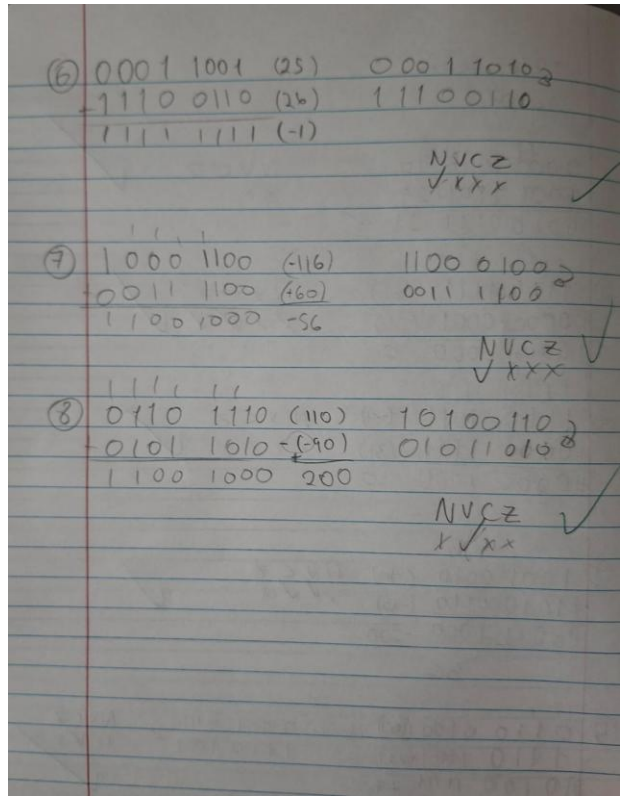


ArrayB



Task 2:





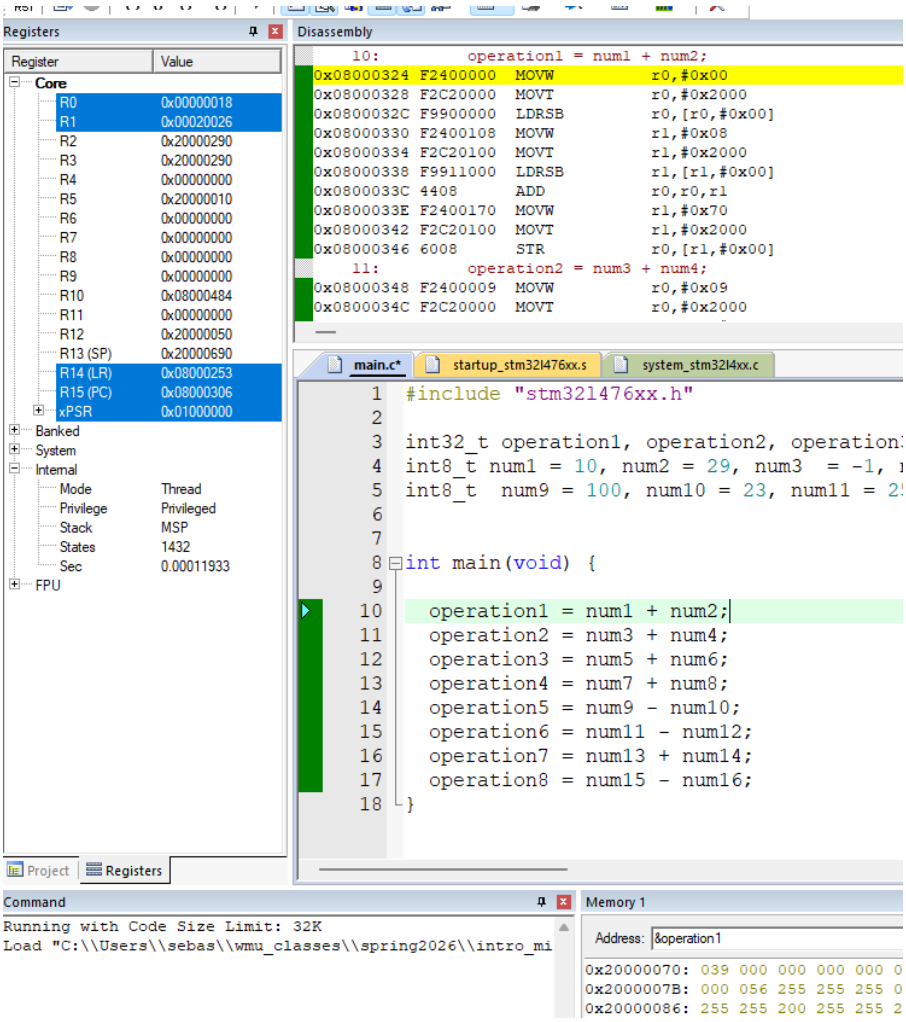
The C Code:

main.c* | startup_stm32l476xx.s | system_stm32l4xx.c

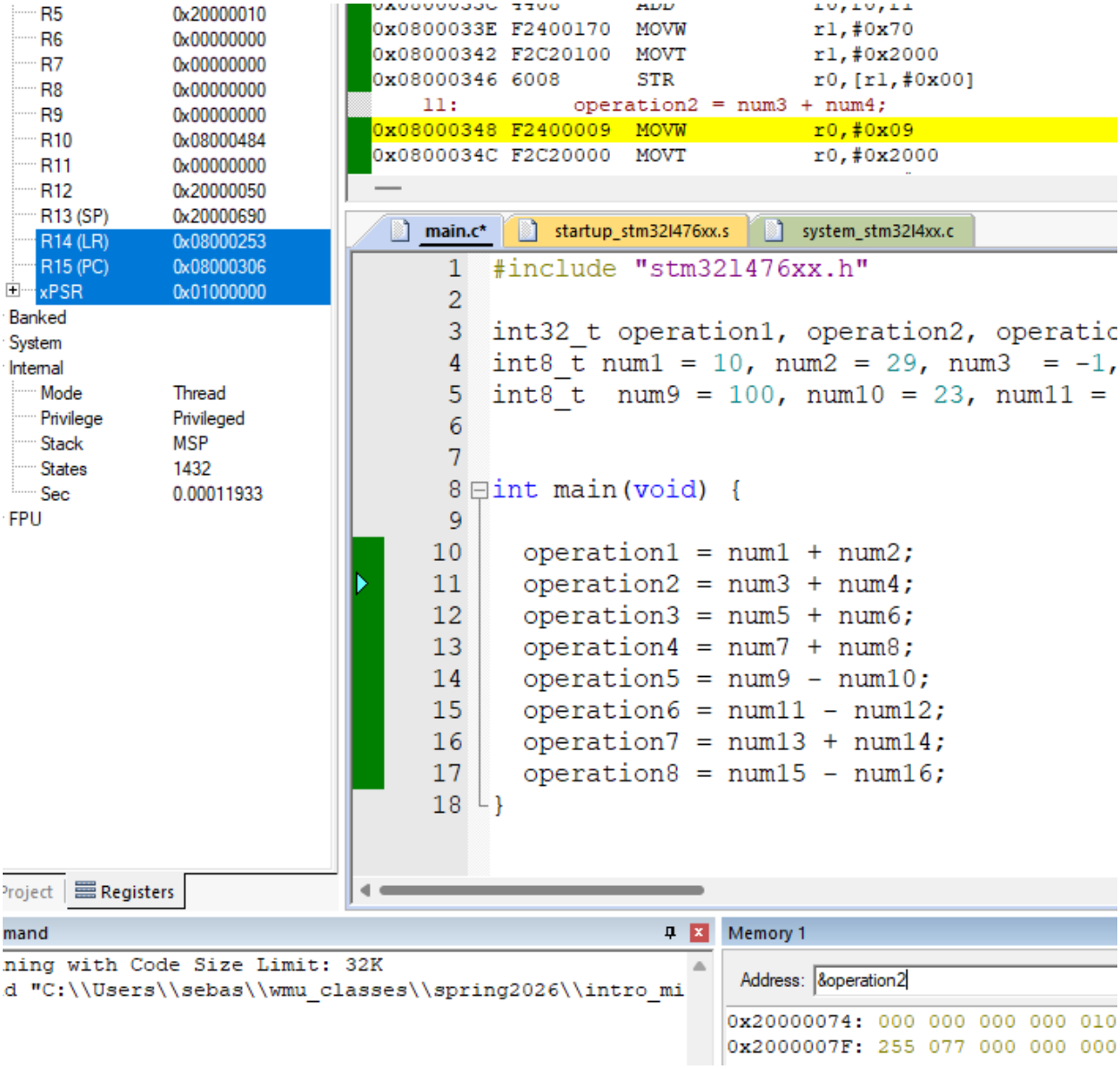
```c
#include "stm32l476xx.h"

int32_t operation1, operation2, operation3, operation4, operation5, operation6, operation7, operation8;
int8_t num1 = 10, num2 = 29, num3  = -1, num4 = 1, num5 = -21, num6 = 31, num7 = -110, num8 = -90;
int8_t  num9 = 100, num10 = 23, num11 = 25, num12 = 26, num13 = -116, num14 = 60, num15 = 110, num16 = -90;


int main(void) {

  operation1 = num1 + num2;
  operation2 = num3 + num4;
  operation3 = num5 + num6;
  operation4 = num7 + num8;
  operation5 = num9 - num10;
  operation6 = num11 - num12;
  operation7 = num13 + num14;
  operation8 = num15 - num16;
}
```

Operations:

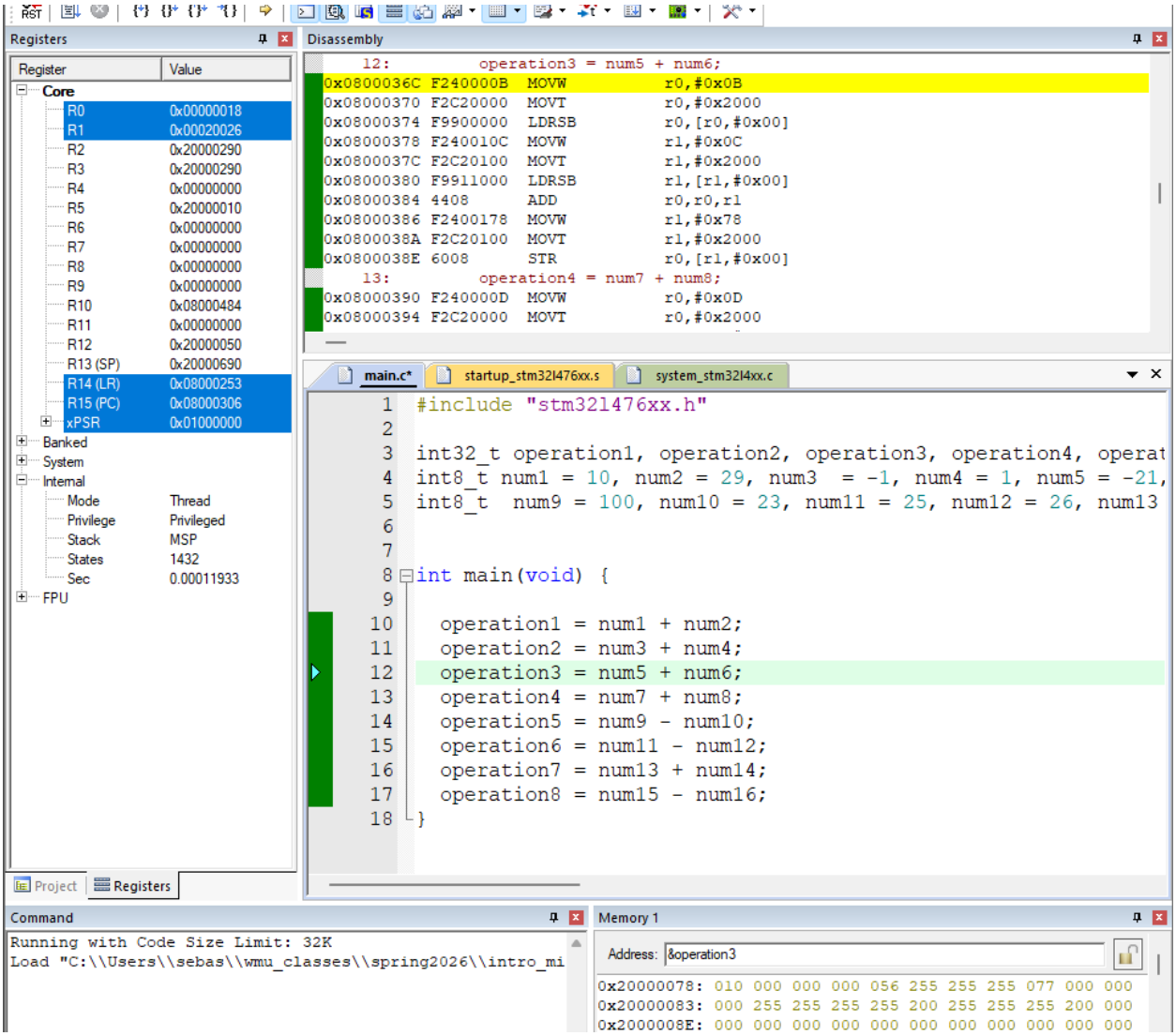
1.



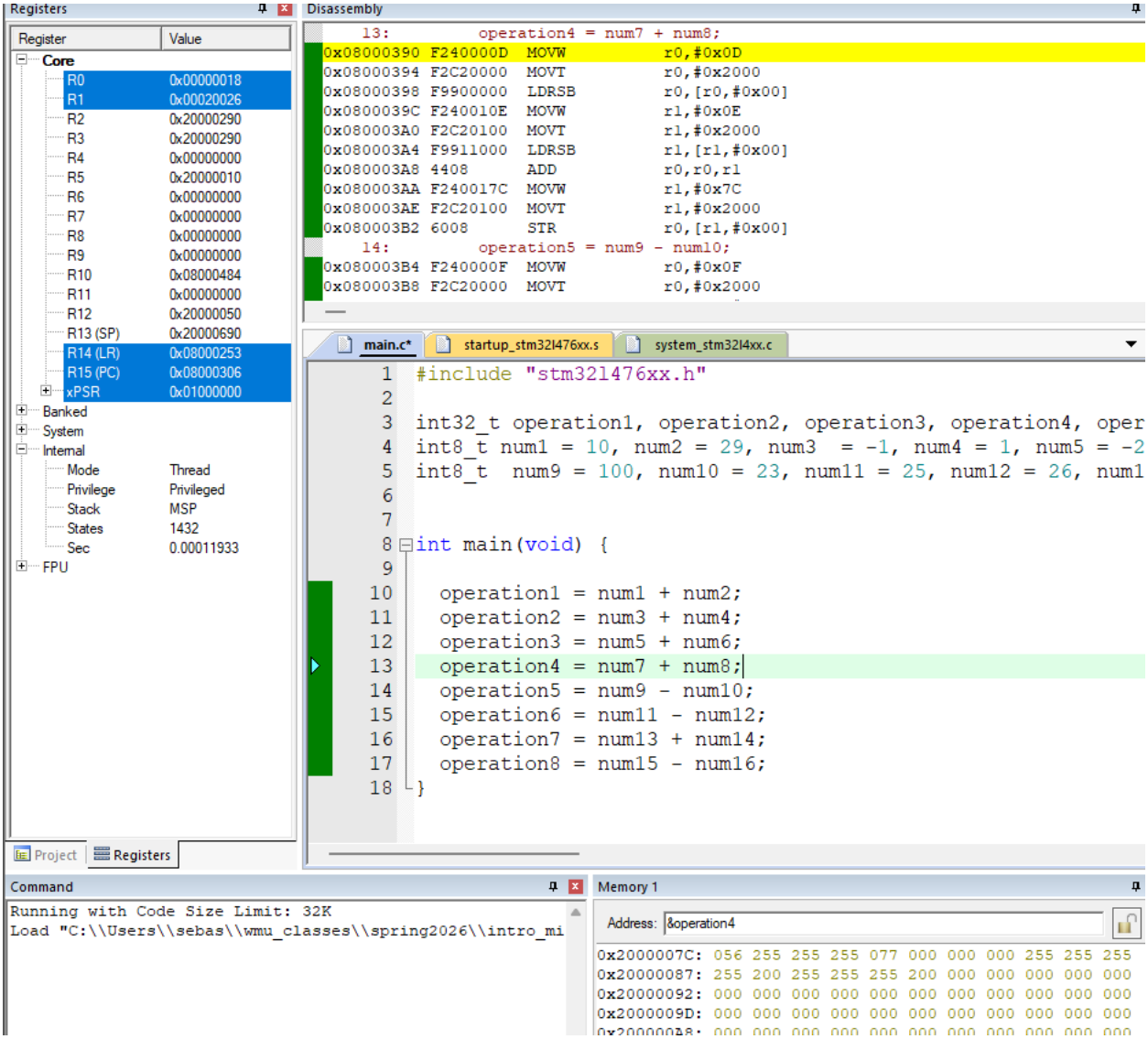
2.


R5 0x20000010
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x08000484
R11 0x00000000
R12 0x20000050
R13 (SP) 0x20000690
R14 (LR) 0x08000253
R15 (PC) 0x08000306
xPSR 0x01000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 1432
Sec 0.00011933
FPU
0x0800033E F2400170 MOVW r1,#0x70
0x08000342 F2C20100 MOVT r1,#0x2000
0x08000346 6008 STR r0,[r1,#0x00]
11: operation2 = num3 + num4;
0x08000348 F2400009 MOVW r0,#0x09
0x0800034C F2C20000 MOVT r0,#0x2000
main.c*
startup_stm32l476xx.s
system_stm32l4xx.c
#include "stm32l476xx.h"
int32_t operation1, operation2, operatic
int8_t num1 = 10, num2 = 29, num3 = -1,
int8_t num9 = 100, num10 = 23, num11 =
int main(void) {
operation1 = num1 + num2;
operation2 = num3 + num4;
operation3 = num5 + num6;
operation4 = num7 + num8;
operation5 = num9 - num10;
operation6 = num11 - num12;
operation7 = num13 + num14;
operation8 = num15 - num16;
}
Registers
mand
ning with Code Size Limit: 32K
d "C:\\Users\\sebas\\wmu_classes\\spring2026\\intro_mi
Memory 1
Address: &operation2
0x20000074: 000 000 000 000 010
0x2000007F: 255 077 000 000 000

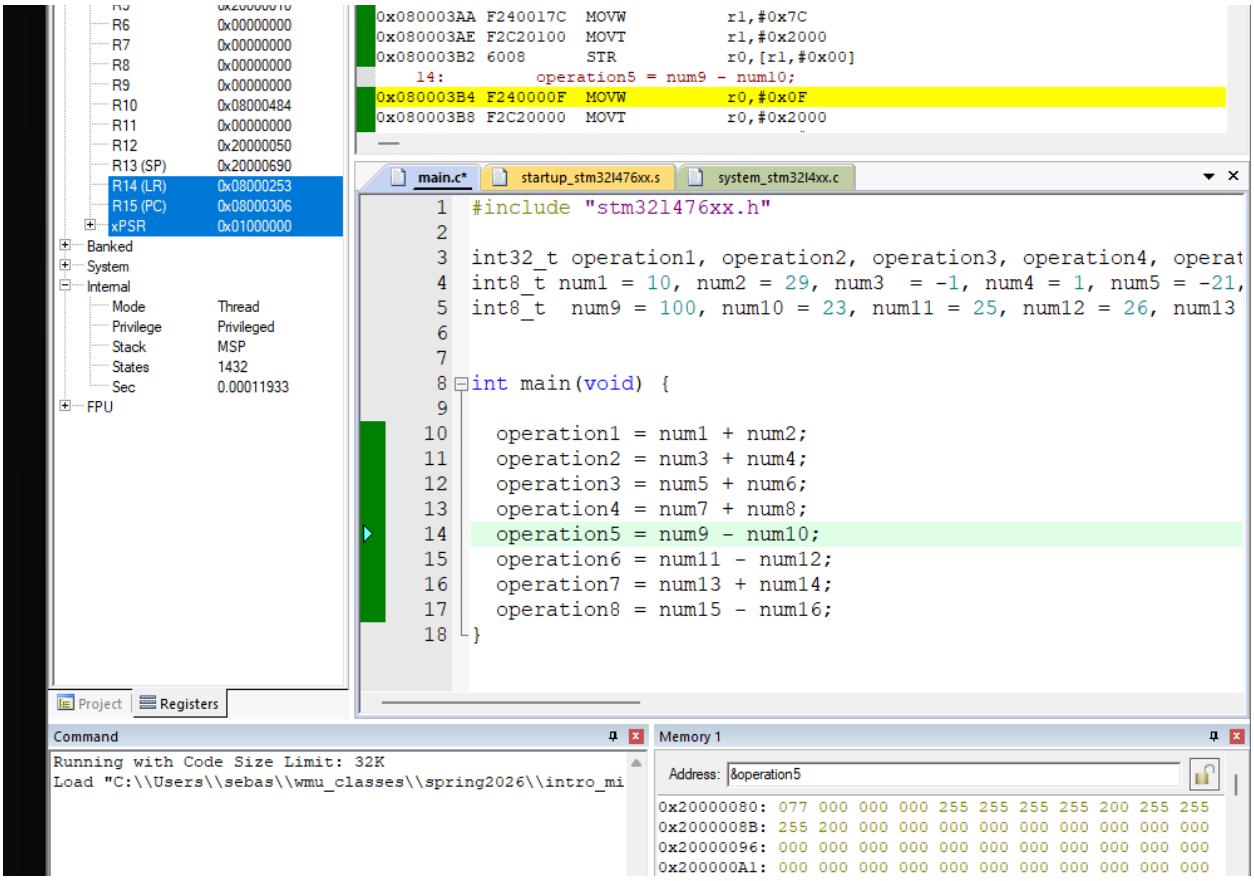
3.


Registers
Register
Value
Core
R0 0x00000018
R1 0x00020026
R2 0x20000290
R3 0x20000290
R4 0x00000000
R5 0x20000010
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x08000484
R11 0x00000000
R12 0x20000050
R13 (SP) 0x20000690
R14 (LR) 0x08000253
R15 (PC) 0x08000306
xPSR 0x01000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 1432
Sec 0.00011933
FPU
Project
Registers
Disassembly
12: operation3 = num5 + num6;
0x0800036C F240000B MOVW r0,#0x0B
0x08000370 F2C20000 MOVT r0,#0x2000
0x08000374 F9900000 LDRSB r0,[r0,#0x00]
0x08000378 F240010C MOVW r1,#0x0C
0x0800037C F2C20100 MOVT r1,#0x2000
0x08000380 F9911000 LDRSB r1,[r1,#0x00]
0x08000384 4408 ADD r0,r0,r1
0x08000386 F2400178 MOVW r1,#0x78
0x0800038A F2C20100 MOVT r1,#0x2000
0x0800038E 6008 STR r0,[r1,#0x00]
13: operation4 = num7 + num8;
0x08000390 F240000D MOVW r0,#0x0D
0x08000394 F2C20000 MOVT r0,#0x2000
main.c*
startup_stm32l476xx.s
system_stm32l4xx.c
#include "stm32l476xx.h"
int32_t operation1, operation2, operation3, operation4, operat
int8_t num1 = 10, num2 = 29, num3 = -1, num4 = 1, num5 = -21,
int8_t num9 = 100, num10 = 23, num11 = 25, num12 = 26, num13
int main(void) {
operation1 = num1 + num2;
operation2 = num3 + num4;
operation3 = num5 + num6;
operation4 = num7 + num8;
operation5 = num9 - num10;
operation6 = num11 - num12;
operation7 = num13 + num14;
operation8 = num15 - num16;
}
Command
Running with Code Size Limit: 32K
Load "C:\\Users\\sebas\\wmu_classes\\spring2026\\intro_mi
Memory 1
Address: &operation3
0x20000078: 010 000 000 000 056 255 255 255 077 000 000
0x20000083: 000 255 255 255 255 200 255 255 255 200 000
0x2000008E: 000 000 000 000 000 000 000 000 000 000 000

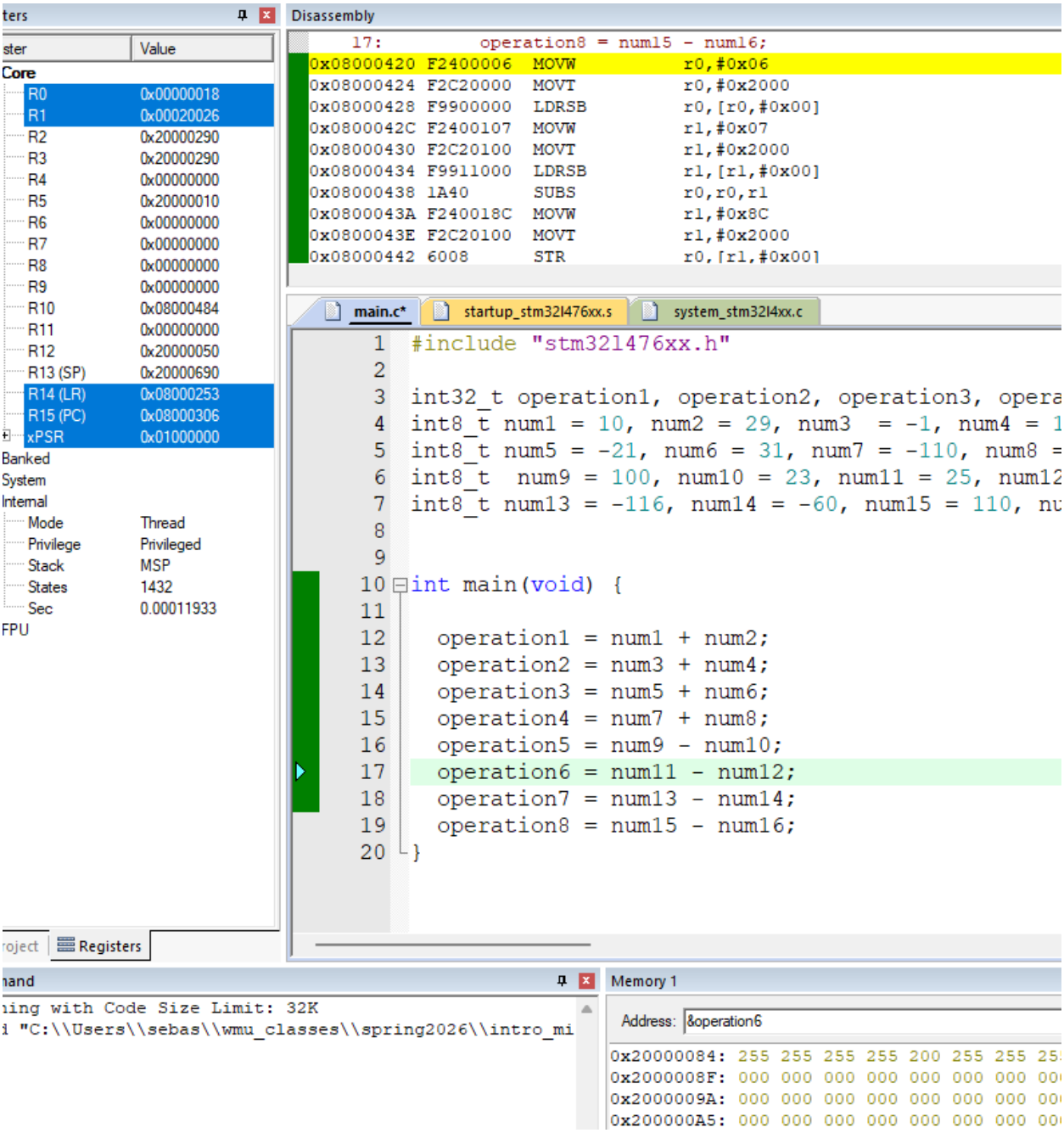
4.


Registers
Register
Value
Core
R0 0x00000018
R1 0x00020026
R2 0x20000290
R3 0x20000290
R4 0x00000000
R5 0x20000010
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x08000484
R11 0x00000000
R12 0x20000050
R13 (SP) 0x20000690
R14 (LR) 0x08000253
R15 (PC) 0x08000306
xPSR 0x01000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 1432
Sec 0.00011933
FPU
Project
Registers
Disassembly
13: operation4 = num7 + num8;
0x08000390 F240000D MOVW r0,#0x0D
0x08000394 F2C20000 MOVT r0,#0x2000
0x08000398 F9900000 LDRSB r0,[r0,#0x00]
0x0800039C F240010E MOVW r1,#0x0E
0x080003A0 F2C20100 MOVT r1,#0x2000
0x080003A4 F9911000 LDRSB r1,[r1,#0x00]
0x080003A8 4408 ADD r0,r0,r1
0x080003AA F240017C MOVW r1,#0x7C
0x080003AE F2C20100 MOVT r1,#0x2000
0x080003B2 6008 STR r0,[r1,#0x00]
14: operation5 = num9 - num10;
0x080003B4 F240000F MOVW r0,#0x0F
0x080003B8 F2C20000 MOVT r0,#0x2000
main.c*
startup_stm32l476xx.s
system_stm32l4xx.c
1 #include "stm32l476xx.h"
2
3 int32_t operation1, operation2, operation3, operation4, oper
4 int8_t num1 = 10, num2 = 29, num3 = -1, num4 = 1, num5 = -2
5 int8_t num9 = 100, num10 = 23, num11 = 25, num12 = 26, num1
6
7
8 int main(void) {
9
10 operation1 = num1 + num2;
11 operation2 = num3 + num4;
12 operation3 = num5 + num6;
13 operation4 = num7 + num8;
14 operation5 = num9 - num10;
15 operation6 = num11 - num12;
16 operation7 = num13 + num14;
17 operation8 = num15 - num16;
18 }
Command
Running with Code Size Limit: 32K
Load "C:\\Users\\sebas\\wmu_classes\\spring2026\\intro_mi
Memory 1
Address: &operation4
0x2000007C: 056 255 255 255 077 000 000 000 255 255 255
0x20000087: 255 200 255 255 255 200 000 000 000 000 000
0x20000092: 000 000 000 000 000 000 000 000 000 000 000
0x2000009D: 000 000 000 000 000 000 000 000 000 000 000

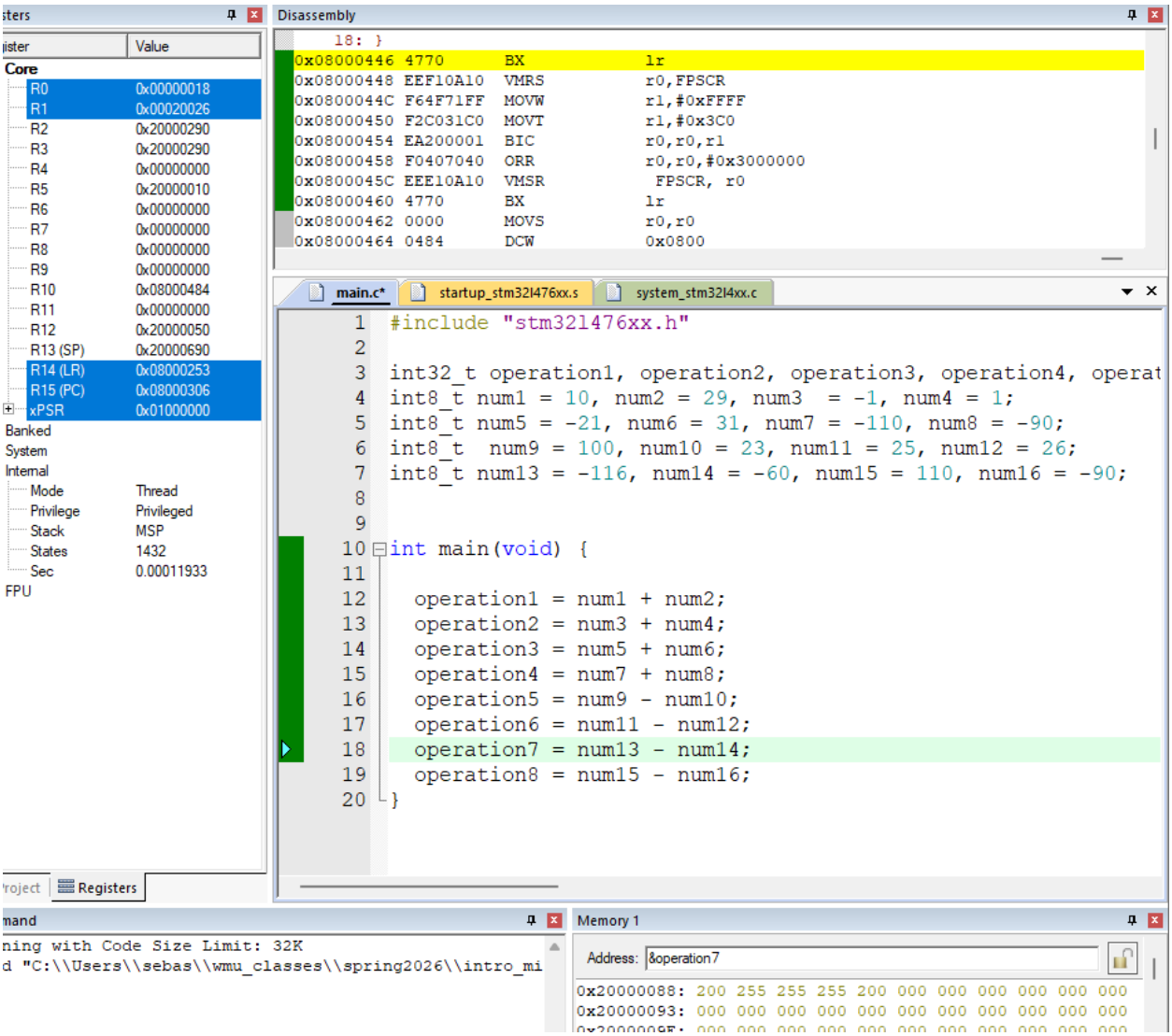
5.


R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x08000484
R11 0x00000000
R12 0x20000050
R13 (SP) 0x20000690
R14 (LR) 0x08000253
R15 (PC) 0x08000306
xPSR 0x01000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 1432
Sec 0.00011933
FPU
Project
Registers
0x080003AA F240017C MOVW r1,#0x7C
0x080003AE F2C20100 MOVT r1,#0x2000
0x080003B2 6008 STR r0,[r1,#0x00]
14: operation5 = num9 - num10;
0x080003B4 F240000F MOVW r0,#0x0F
0x080003B8 F2C20000 MOVT r0,#0x2000
main.c*
startup_stm32l476xx.s
system_stm32l4xx.c
#include "stm32l476xx.h"
int32_t operation1, operation2, operation3, operation4, operat
int8_t num1 = 10, num2 = 29, num3 = -1, num4 = 1, num5 = -21,
int8_t num9 = 100, num10 = 23, num11 = 25, num12 = 26, num13
int main(void) {
operation1 = num1 + num2;
operation2 = num3 + num4;
operation3 = num5 + num6;
operation4 = num7 + num8;
operation5 = num9 - num10;
operation6 = num11 - num12;
operation7 = num13 + num14;
operation8 = num15 - num16;
}
Command
Running with Code Size Limit: 32K
Load "C:\\Users\\sebas\\wmu_classes\\spring2026\\intro_mi
Memory 1
Address: &operation5
0x20000080: 077 000 000 000 255 255 255 255 200 255 255
0x2000008B: 255 200 000 000 000 000 000 000 000 000 000
0x20000096: 000 000 000 000 000 000 000 000 000 000 000
0x200000A1: 000 000 000 000 000 000 000 000 000 000 000

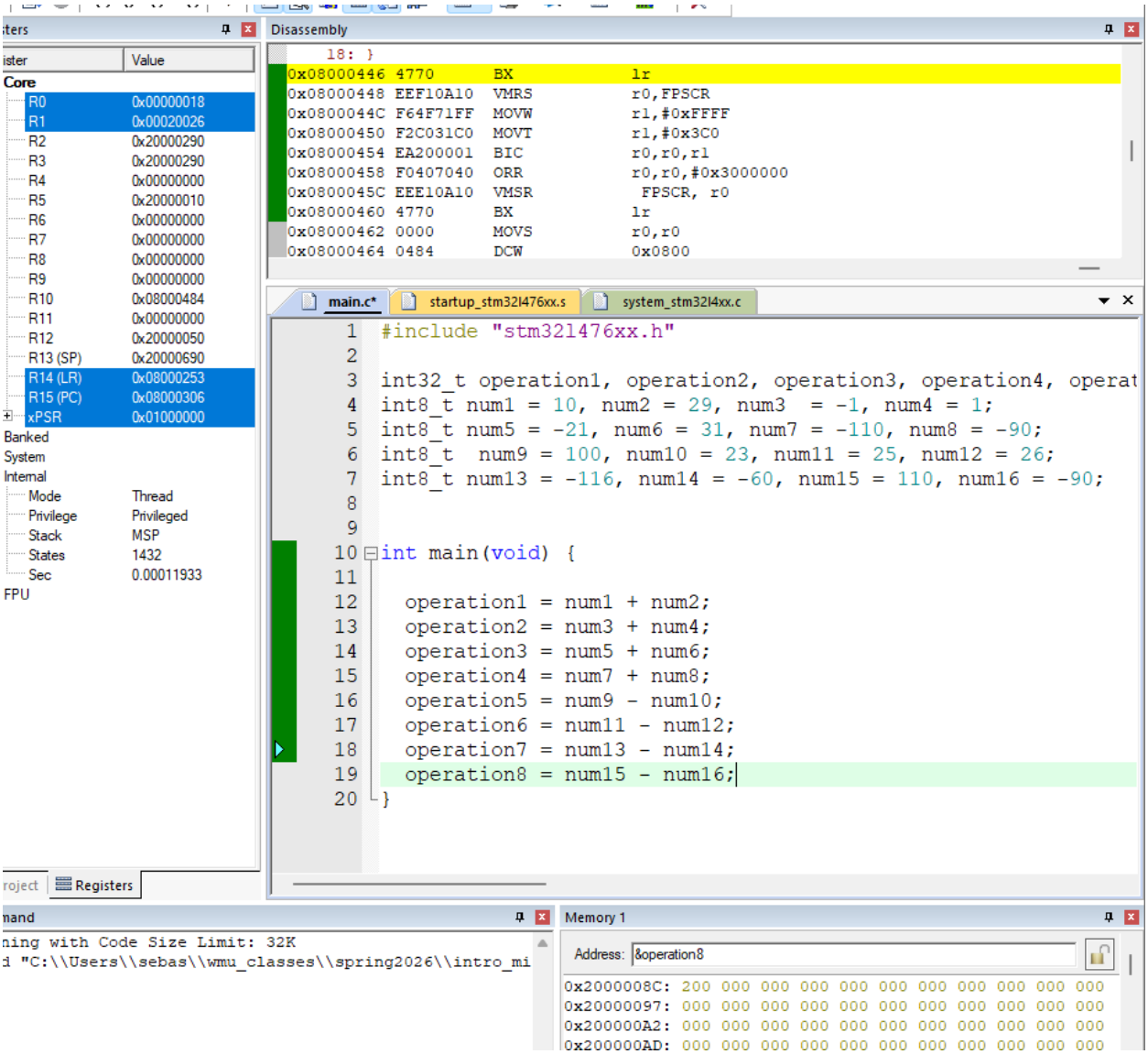
6.


ters
ster
Value
Core
R0 0x00000018
R1 0x00020026
R2 0x20000290
R3 0x20000290
R4 0x00000000
R5 0x20000010
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x08000484
R11 0x00000000
R12 0x20000050
R13 (SP) 0x20000690
R14 (LR) 0x08000253
R15 (PC) 0x08000306
xPSR 0x01000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 1432
Sec 0.00011933
FPU
roject
Registers
Disassembly
17: operation8 = num15 - num16;
0x08000420 F2400006 MOVW r0,#0x06
0x08000424 F2C20000 MOVT r0,#0x2000
0x08000428 F9900000 LDRSB r0,[r0,#0x00]
0x0800042C F2400107 MOVW r1,#0x07
0x08000430 F2C20100 MOVT r1,#0x2000
0x08000434 F9911000 LDRSB r1,[r1,#0x00]
0x08000438 1A40 SUBS r0,r0,r1
0x0800043A F240018C MOVW r1,#0x8C
0x0800043E F2C20100 MOVT r1,#0x2000
0x08000442 6008 STR r0,[r1,#0x00]
main.c*
startup_stm32l476xx.s
system_stm32l4xx.c
1 #include "stm32l476xx.h"
2
3 int32_t operation1, operation2, operation3, opera
4 int8_t num1 = 10, num2 = 29, num3 = -1, num4 = 1
5 int8_t num5 = -21, num6 = 31, num7 = -110, num8 =
6 int8_t num9 = 100, num10 = 23, num11 = 25, num12
7 int8_t num13 = -116, num14 = -60, num15 = 110, nu
8
9
10 int main(void) {
11
12 operation1 = num1 + num2;
13 operation2 = num3 + num4;
14 operation3 = num5 + num6;
15 operation4 = num7 + num8;
16 operation5 = num9 - num10;
17 operation6 = num11 - num12;
18 operation7 = num13 - num14;
19 operation8 = num15 - num16;
20 }
nand
ning with Code Size Limit: 32K
l "C:\\Users\\sebas\\wmu_classes\\spring2026\\intro_mi
Memory 1
Address: &operation6
0x20000084: 255 255 255 255 200 255 255 25
0x2000008F: 000 000 000 000 000 000 000 00
0x2000009A: 000 000 000 000 000 000 000 00
0x200000A5: 000 000 000 000 000 000 000 00

7.

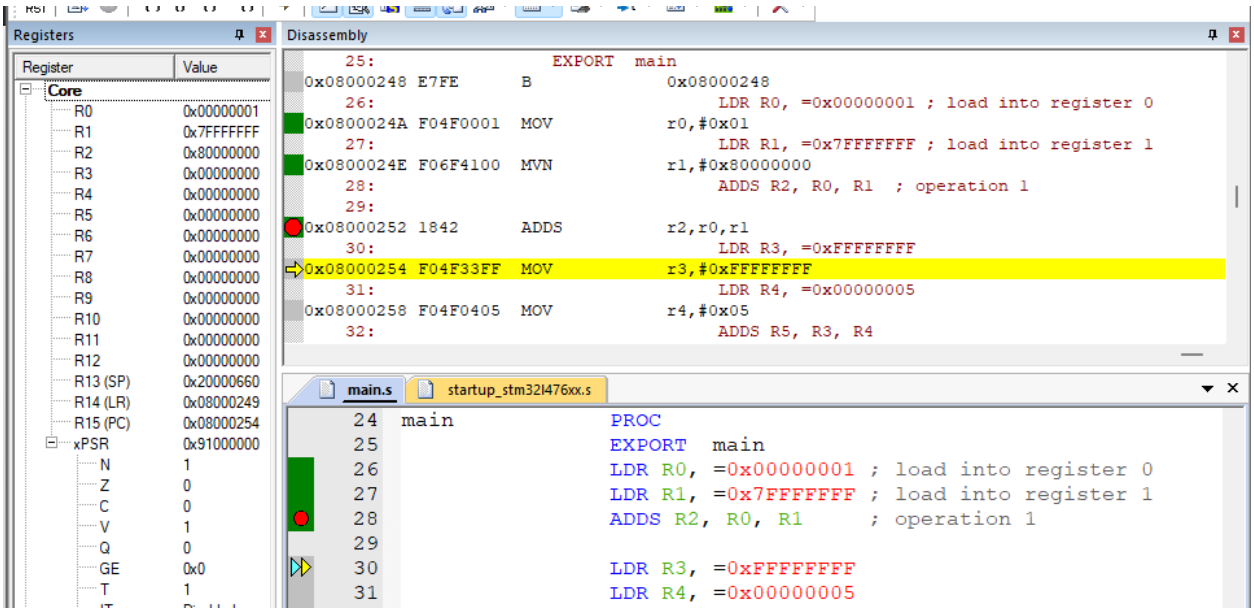

Disassembly
18: }
0x08000446 4770 BX lr
0x08000448 EEF10A10 VMRS r0,FPSCR
0x0800044C F64F71FF MOVW r1,#0xFFFF
0x08000450 F2C031C0 MOVT r1,#0x3C0
0x08000454 EA200001 BIC r0,r0,r1
0x08000458 F0407040 ORR r0,r0,#0x3000000
0x0800045C EEE10A10 VMSR FPSCR, r0
0x08000460 4770 BX lr
0x08000462 0000 MOVS r0,r0
0x08000464 0484 DCW 0x0800
Core
R0 0x00000018
R1 0x00020026
R2 0x20000290
R3 0x20000290
R4 0x00000000
R5 0x20000010
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x08000484
R11 0x00000000
R12 0x20000050
R13 (SP) 0x20000690
R14 (LR) 0x08000253
R15 (PC) 0x08000306
xPSR 0x01000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 1432
Sec 0.00011933
FPU
Registers
main.c*
startup_stm32l476xx.s
system_stm32l4xx.c
#include "stm32l476xx.h"
int32_t operation1, operation2, operation3, operation4, operat
int8_t num1 = 10, num2 = 29, num3 = -1, num4 = 1;
int8_t num5 = -21, num6 = 31, num7 = -110, num8 = -90;
int8_t num9 = 100, num10 = 23, num11 = 25, num12 = 26;
int8_t num13 = -116, num14 = -60, num15 = 110, num16 = -90;
int main(void) {
operation1 = num1 + num2;
operation2 = num3 + num4;
operation3 = num5 + num6;
operation4 = num7 + num8;
operation5 = num9 - num10;
operation6 = num11 - num12;
operation7 = num13 - num14;
operation8 = num15 - num16;
}
Memory 1
Address: &operation7
0x20000088: 200 255 255 255 200 000 000 000 000 000 000
0x20000093: 000 000 000 000 000 000 000 000 000 000 000

8.


Disassembly
18: }
0x08000446 4770 BX lr
0x08000448 EEF10A10 VMRS r0,FPSCR
0x0800044C F64F71FF MOVW r1,#0xFFFF
0x08000450 F2C031C0 MOVT r1,#0x3C0
0x08000454 EA200001 BIC r0,r0,r1
0x08000458 F0407040 ORR r0,r0,#0x3000000
0x0800045C EEE10A10 VMSR FPSCR, r0
0x08000460 4770 BX lr
0x08000462 0000 MOVS r0,r0
0x08000464 0484 DCW 0x0800
Core
R0 0x00000018
R1 0x00020026
R2 0x20000290
R3 0x20000290
R4 0x00000000
R5 0x20000010
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x08000484
R11 0x00000000
R12 0x20000050
R13 (SP) 0x20000690
R14 (LR) 0x08000253
R15 (PC) 0x08000306
xPSR 0x01000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 1432
Sec 0.00011933
FPU
Registers
main.c*
startup_stm32l476xx.s
system_stm32l4xx.c
#include "stm32l476xx.h"
int32_t operation1, operation2, operation3, operation4, operat
int8_t num1 = 10, num2 = 29, num3 = -1, num4 = 1;
int8_t num5 = -21, num6 = 31, num7 = -110, num8 = -90;
int8_t num9 = 100, num10 = 23, num11 = 25, num12 = 26;
int8_t num13 = -116, num14 = -60, num15 = 110, num16 = -90;
int main(void) {
operation1 = num1 + num2;
operation2 = num3 + num4;
operation3 = num5 + num6;
operation4 = num7 + num8;
operation5 = num9 - num10;
operation6 = num11 - num12;
operation7 = num13 - num14;
operation8 = num15 - num16;
}
ning with Code Size Limit: 32K
d "C:\\Users\\sebas\\wmu_classes\\spring2026\\intro_mi
Memory 1
Address: &operation8
0x2000008C: 200 000 000 000 000 000 000 000 000 000 000
0x20000097: 000 000 000 000 000 000 000 000 000 000 000
0x200000A2: 000 000 000 000 000 000 000 000 000 000 000
0x200000AD: 000 000 000 000 000 000 000 000 000 000 000

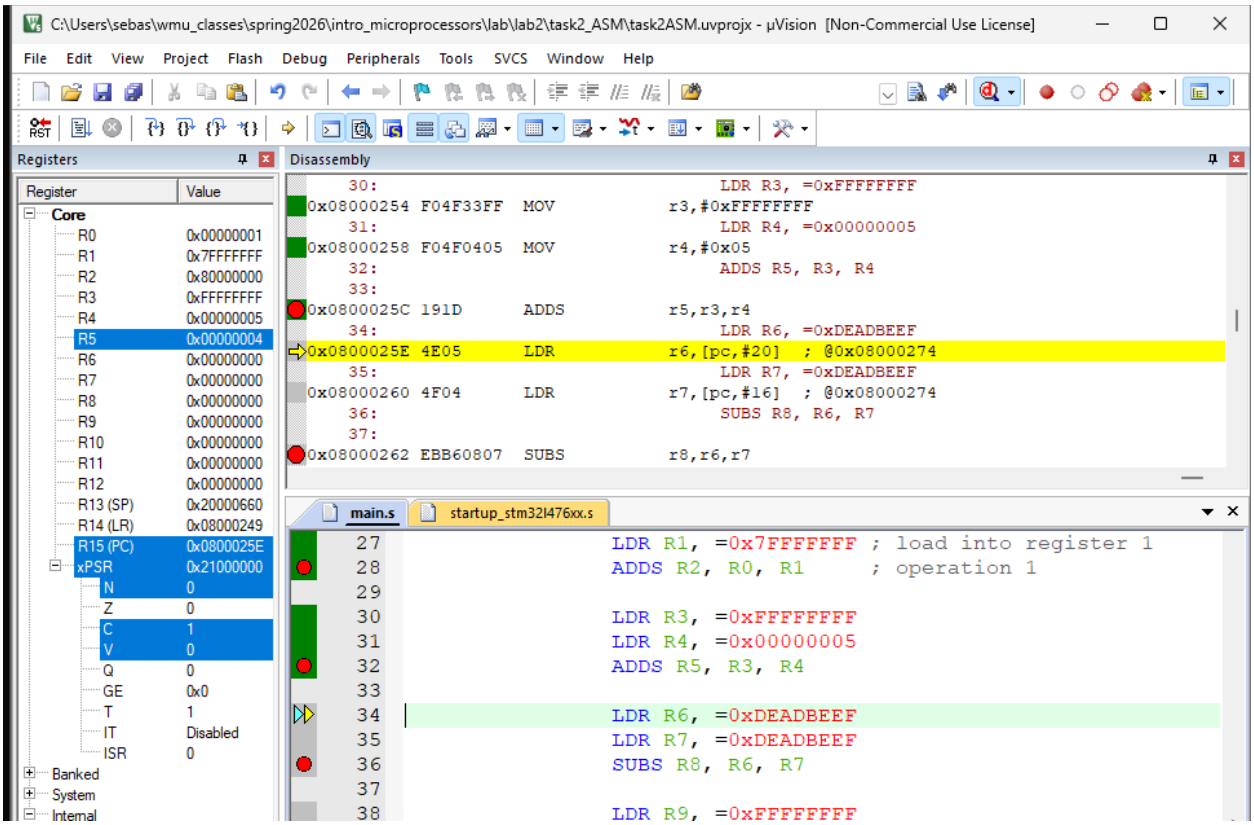
Task 2: Assembly hex operations

OPERATION 1



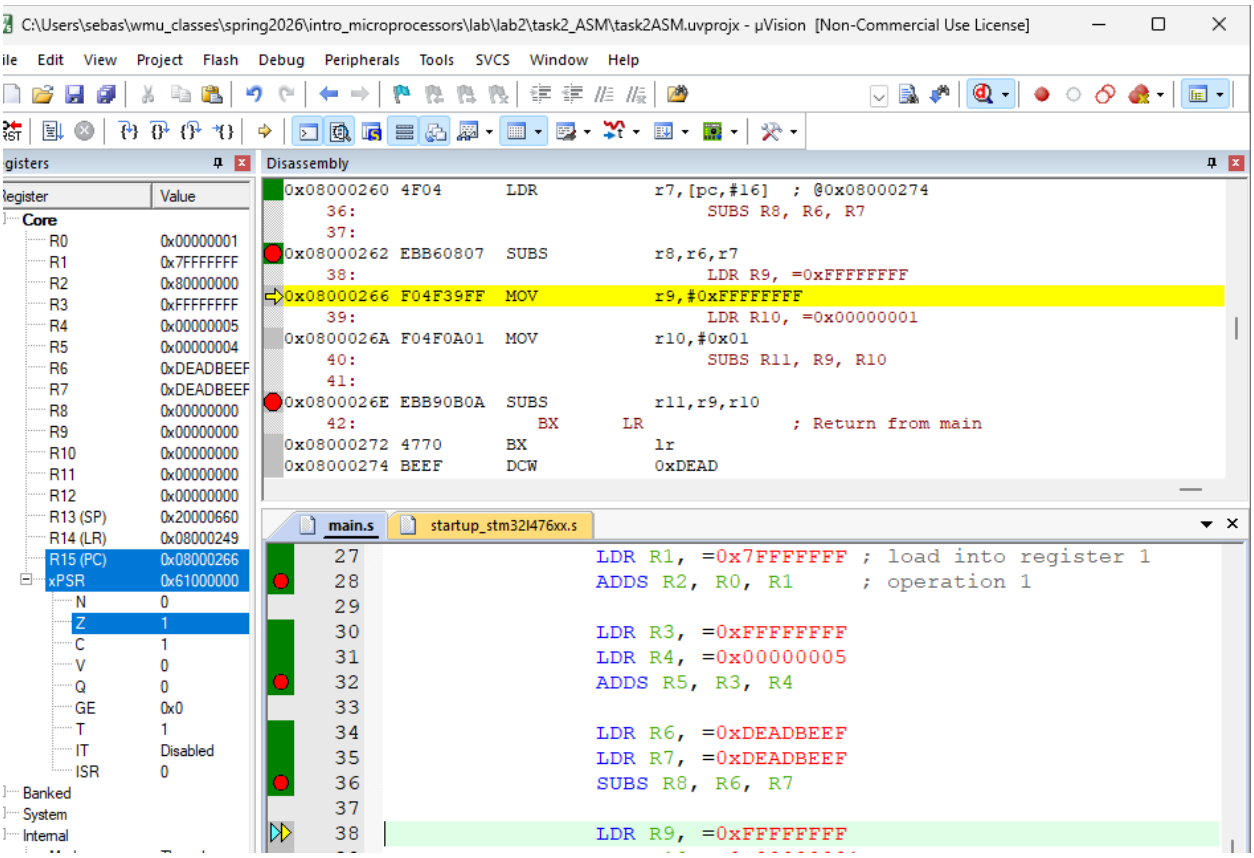
The result makes sense as I added two positive numbers and got a negative result (0x80000000) which has a 1 in its MSB.

OPERATION 2



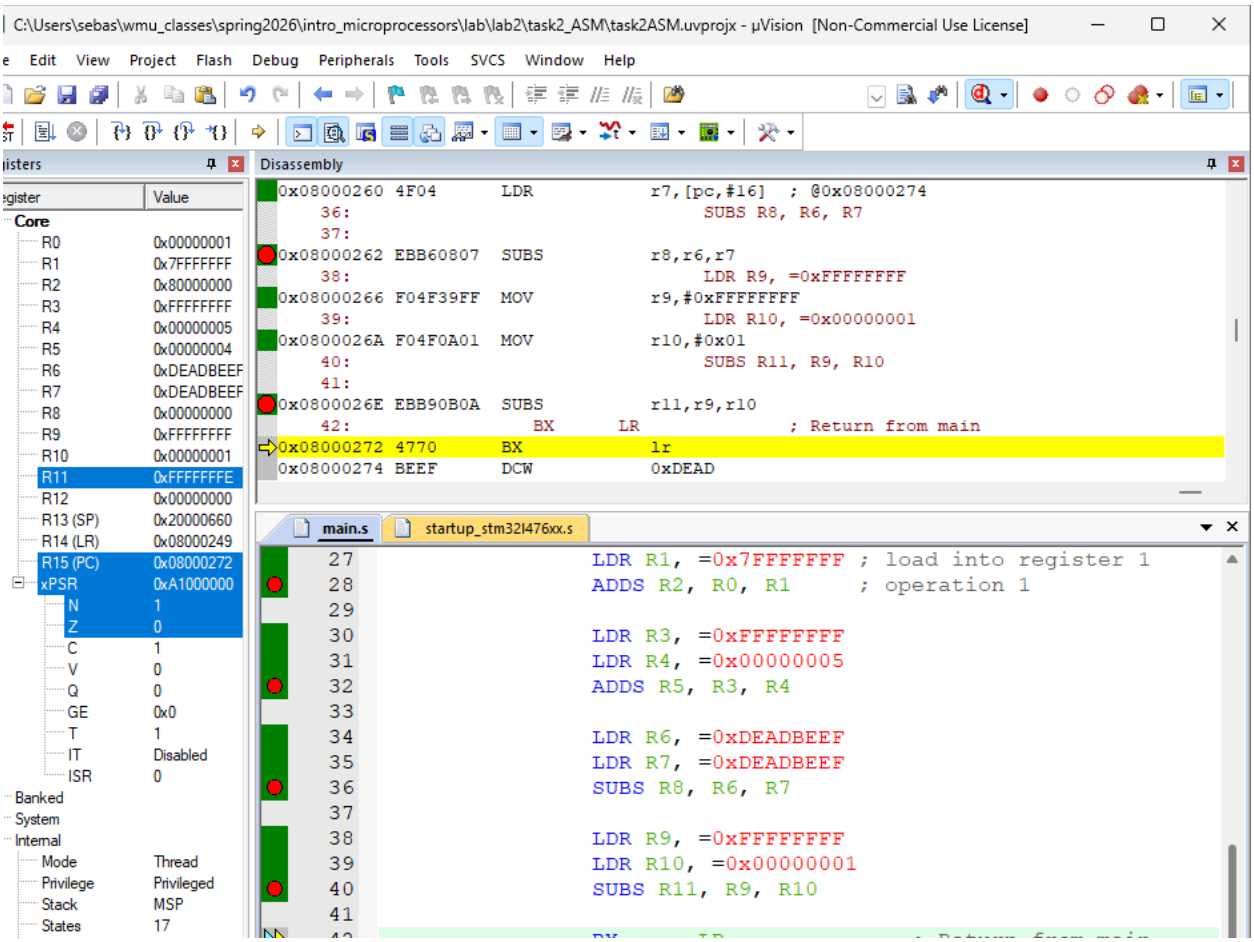
In signed terms, this operation is equal to -1 + 5 = 4, so it makes sense and because I added 5 to to 0xFFFFFFFF it wraps around with a carry.

OPERATION 3



It makes sense because I am subtracting the whole value from itself so the result is exactly zero.

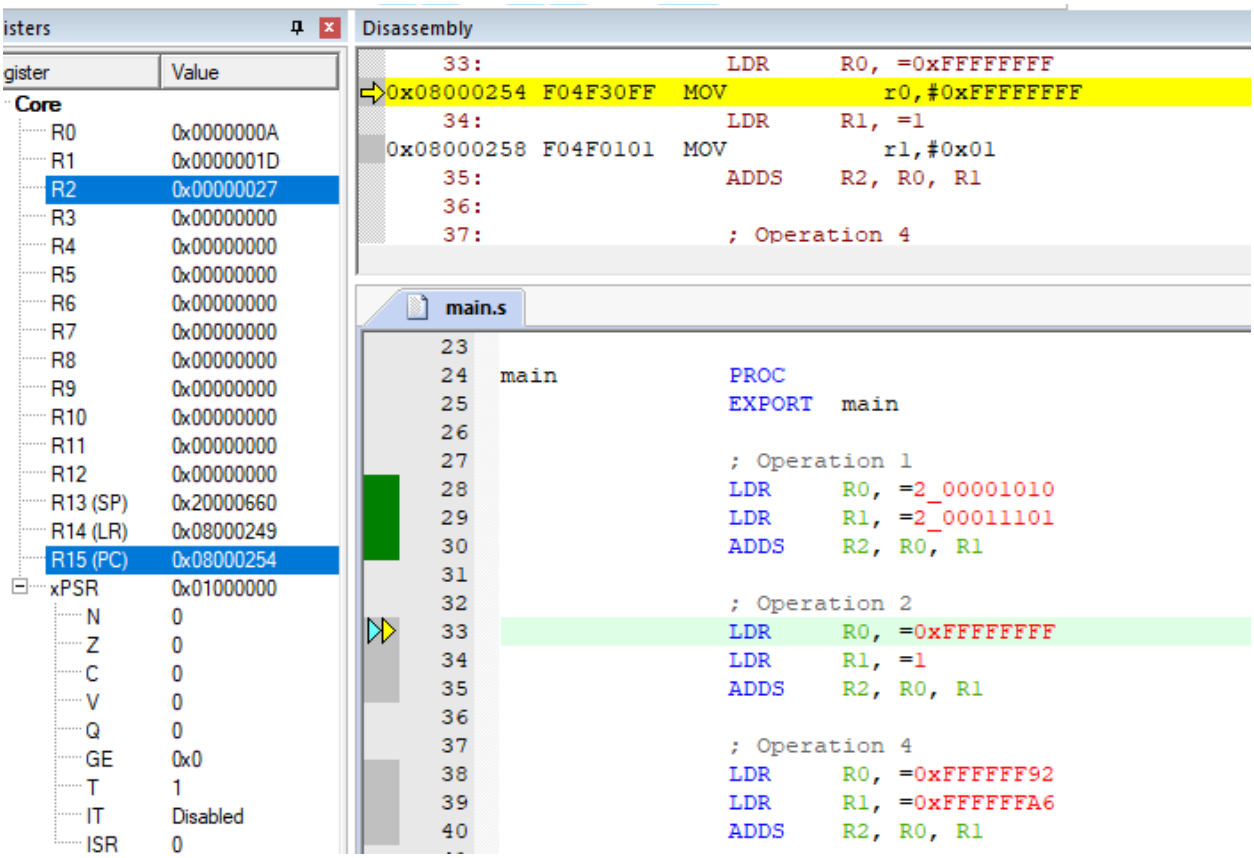
OPERATION 4



It makes sens because the MSB of 0xFFFFFFFE is 1

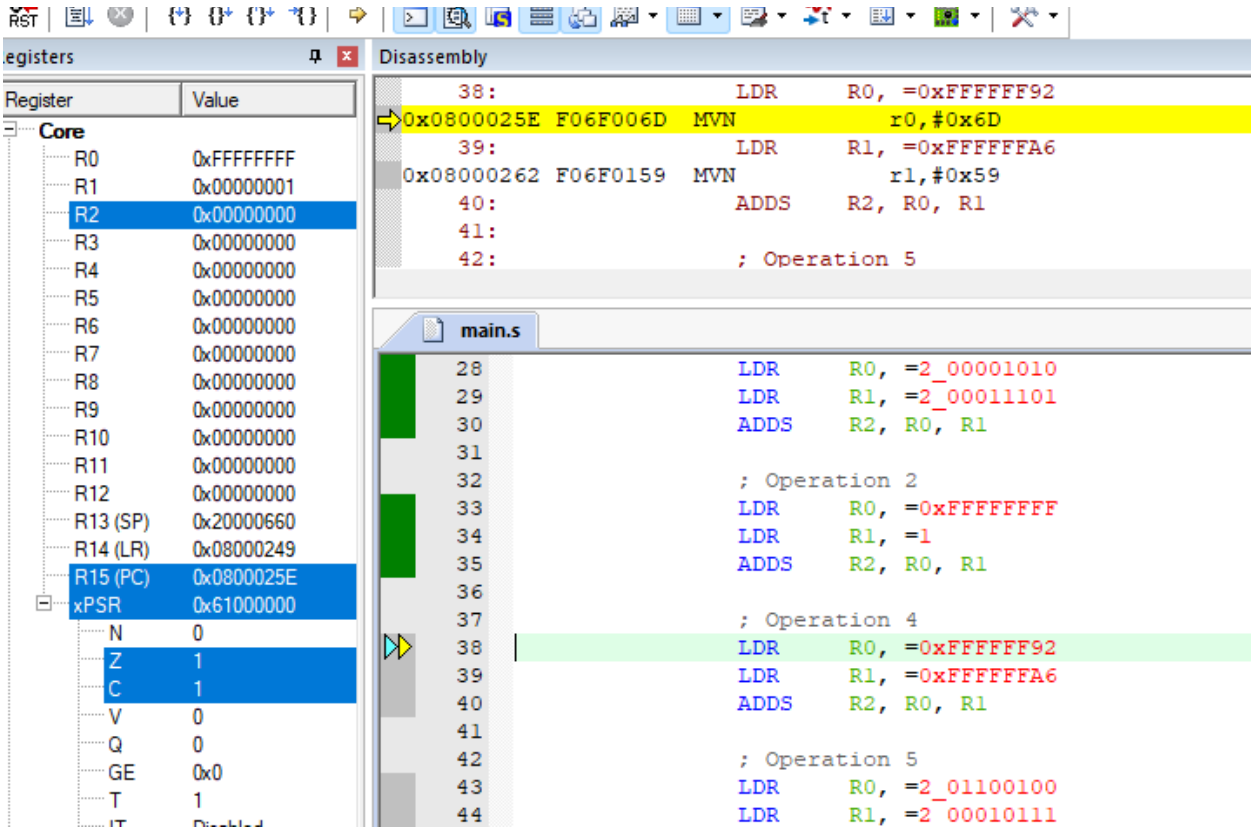
Task 3:

Operation 1:

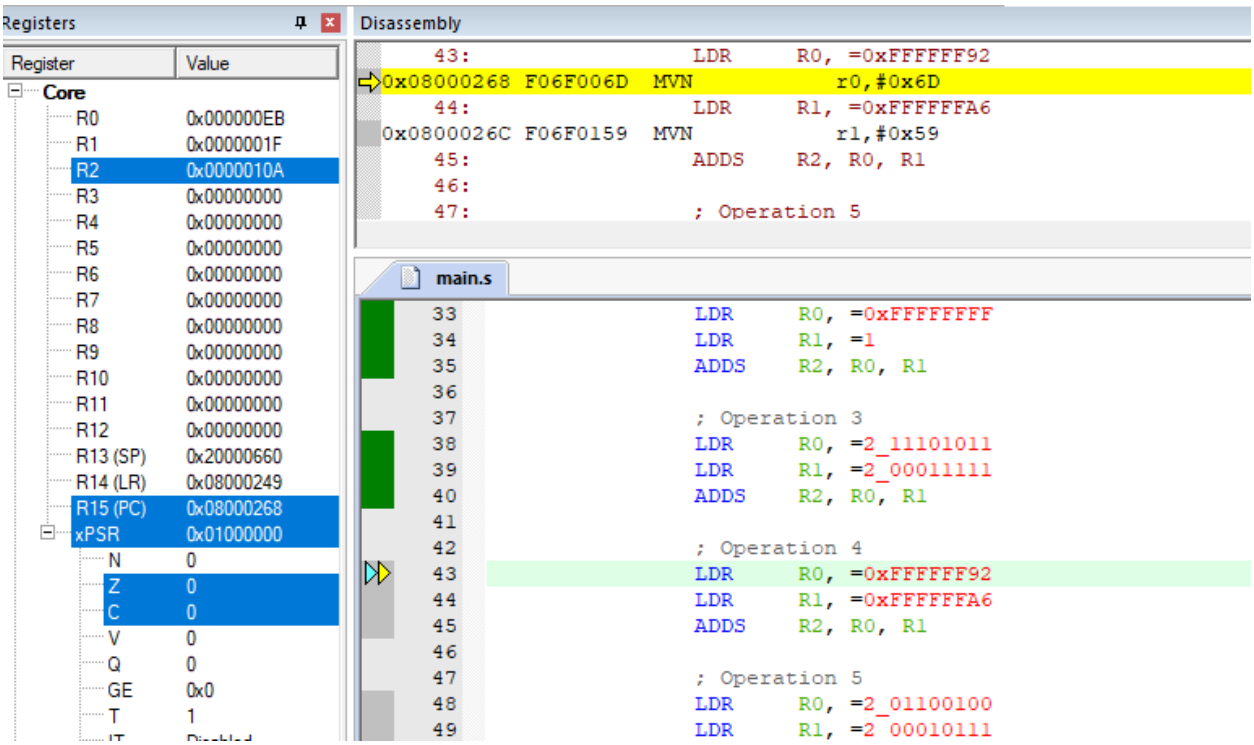


No flags

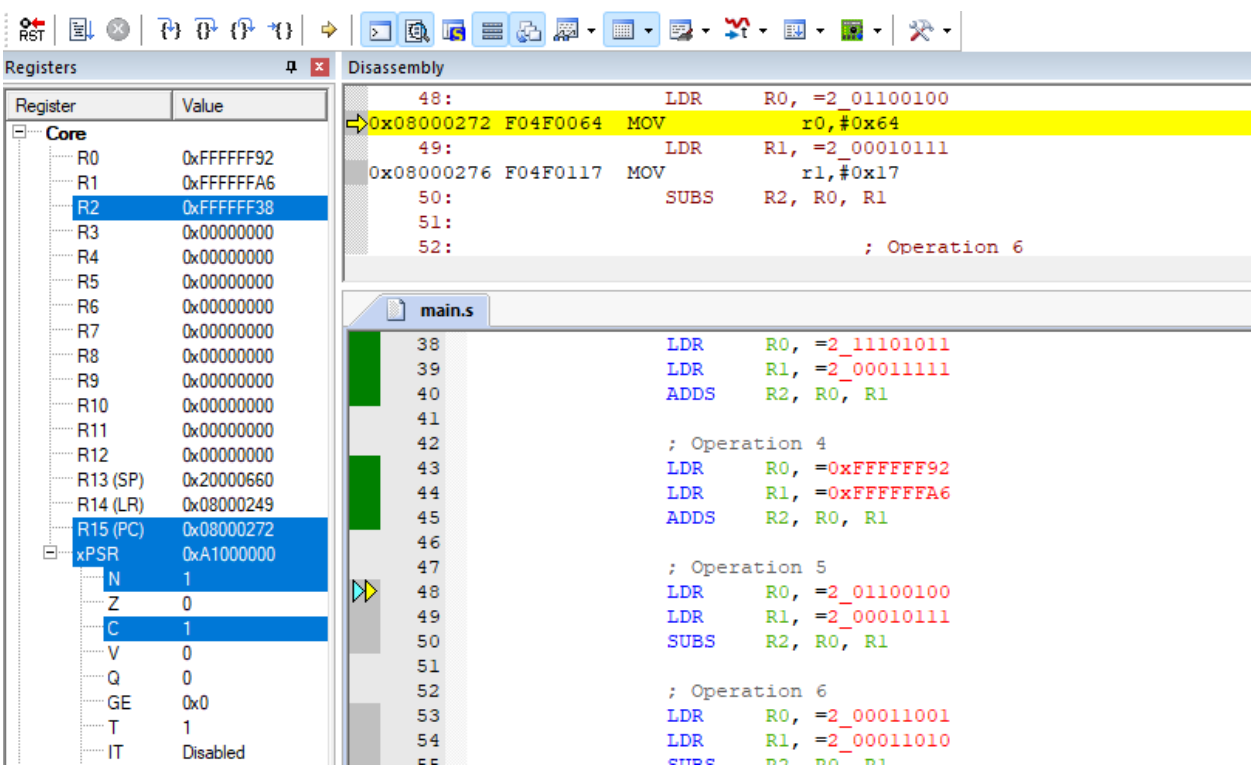
Operation 2:



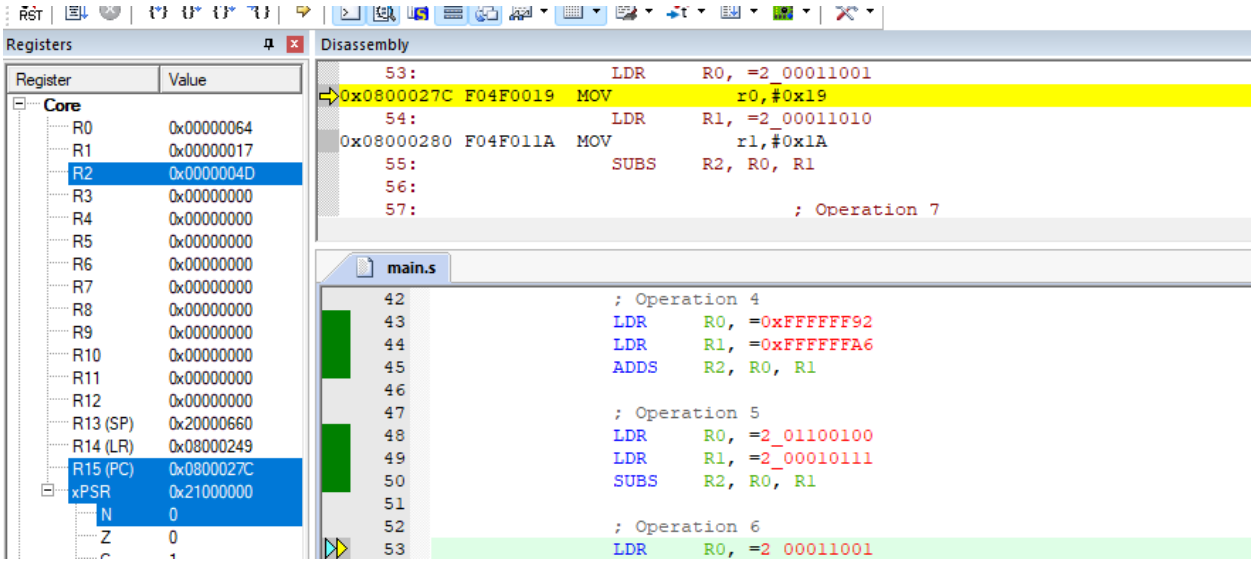
Operation 3:



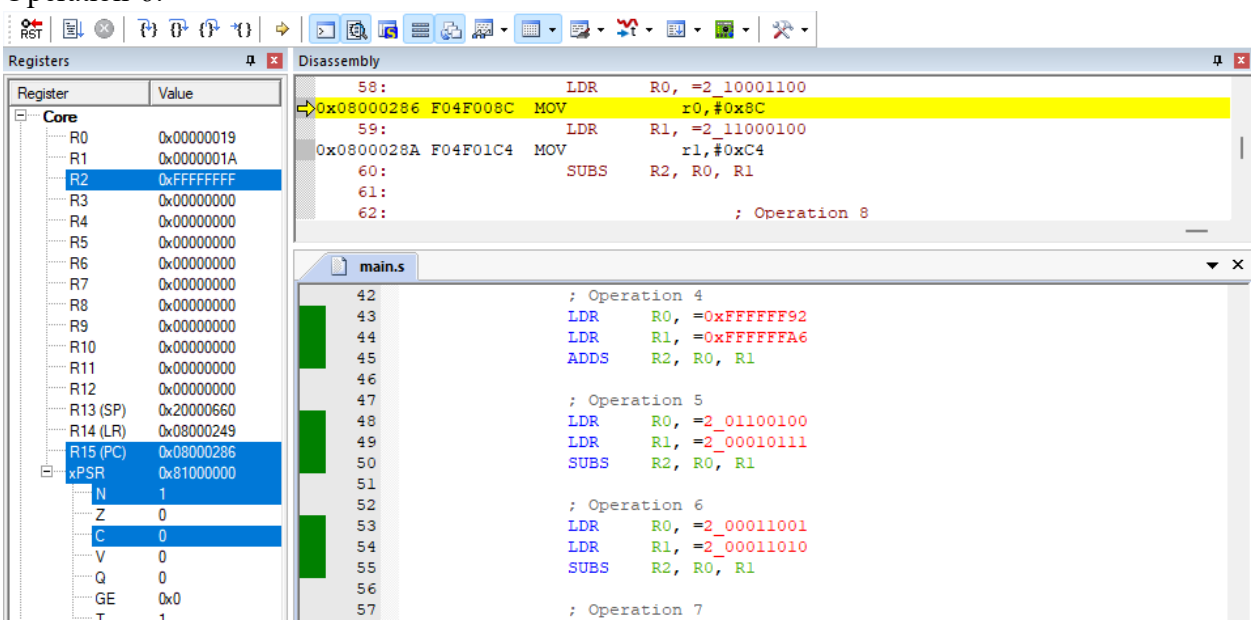
Operation 4:



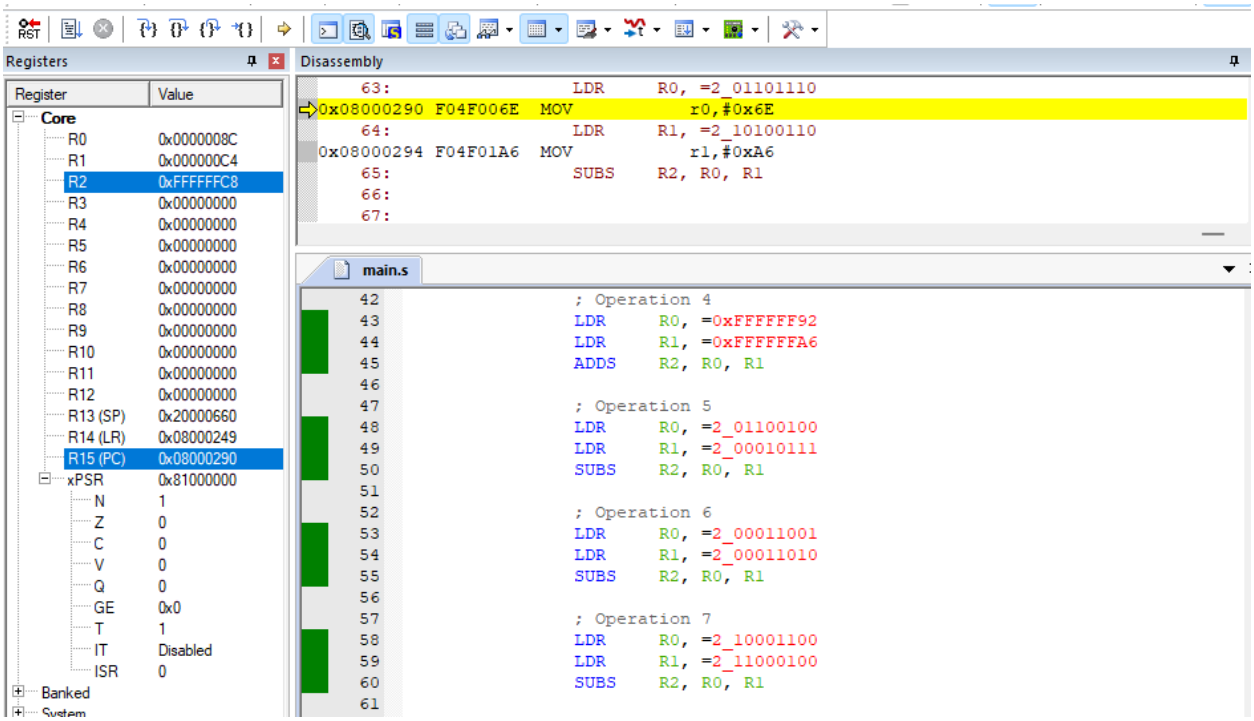
Operation 5:



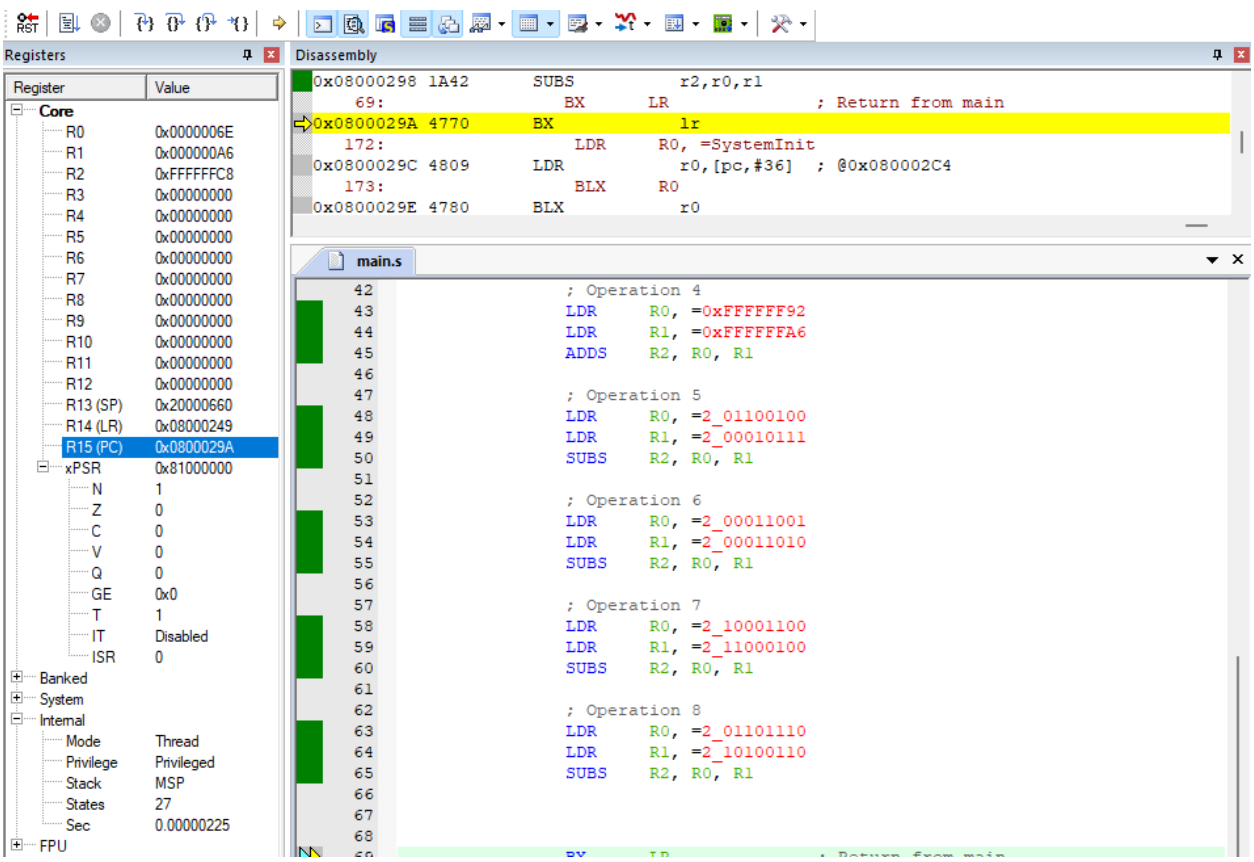
Operation 6:



Operation 7:


Registers
Register
Value
Core
R0 0x0000008C
R1 0x000000C4
R2 0xFFFFFFC8
R3 0x00000000
R4 0x00000000
R5 0x00000000
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x00000000
R11 0x00000000
R12 0x00000000
R13 (SP) 0x20000660
R14 (LR) 0x08000249
R15 (PC) 0x08000290
xPSR 0x81000000
N 1
Z 0
C 0
V 0
Q 0
GE 0x0
T 1
IT Disabled
ISR 0
Banked
Disassembly
63: LDR R0, =2_01101110
0x08000290 F04F006E MOV r0,#0x6E
64: LDR R1, =2_10100110
0x08000294 F04F01A6 MOV r1,#0xA6
65: SUBS R2, R0, R1
66:
67:
main.s
42 ; Operation 4
43 LDR R0, =0xFFFFFF92
44 LDR R1, =0xFFFFFFA6
45 ADDS R2, R0, R1
46
47 ; Operation 5
48 LDR R0, =2_01100100
49 LDR R1, =2_00010111
50 SUBS R2, R0, R1
51
52 ; Operation 6
53 LDR R0, =2_00011001
54 LDR R1, =2_00011010
55 SUBS R2, R0, R1
56
57 ; Operation 7
58 LDR R0, =2_10001100
59 LDR R1, =2_11000100
60 SUBS R2, R0, R1
61

Operation 8:


Registers
Register
Value
Core
R0 0x0000006E
R1 0x000000A6
R2 0xFFFFFFC8
R3 0x00000000
R4 0x00000000
R5 0x00000000
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x00000000
R11 0x00000000
R12 0x00000000
R13 (SP) 0x20000660
R14 (LR) 0x08000249
R15 (PC) 0x0800029A
xPSR 0x81000000
N 1
Z 0
C 0
V 0
Q 0
GE 0x0
T 1
IT Disabled
ISR 0
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 27
Sec 0.00000225
FPU
Disassembly
0x08000298 1A42 SUBS r2,r0,r1
69: BX LR ; Return from main
0x0800029A 4770 BX lr
172: LDR R0, =SystemInit
0x0800029C 4809 LDR r0,[pc,#36] ; @0x080002C4
173: BLX R0
0x0800029E 4780 BLX r0
main.s
42 ; Operation 4
43 LDR R0, =0xFFFFFF92
44 LDR R1, =0xFFFFFFA6
45 ADDS R2, R0, R1
46
47 ; Operation 5
48 LDR R0, =2_01100100
49 LDR R1, =2_00010111
50 SUBS R2, R0, R1
51
52 ; Operation 6
53 LDR R0, =2_00011001
54 LDR R1, =2_00011010
55 SUBS R2, R0, R1
56
57 ; Operation 7
58 LDR R0, =2_10001100
59 LDR R1, =2_11000100
60 SUBS R2, R0, R1
61
62 ; Operation 8
63 LDR R0, =2_01101110
64 LDR R1, =2_10100110
65 SUBS R2, R0, R1
66
67
68

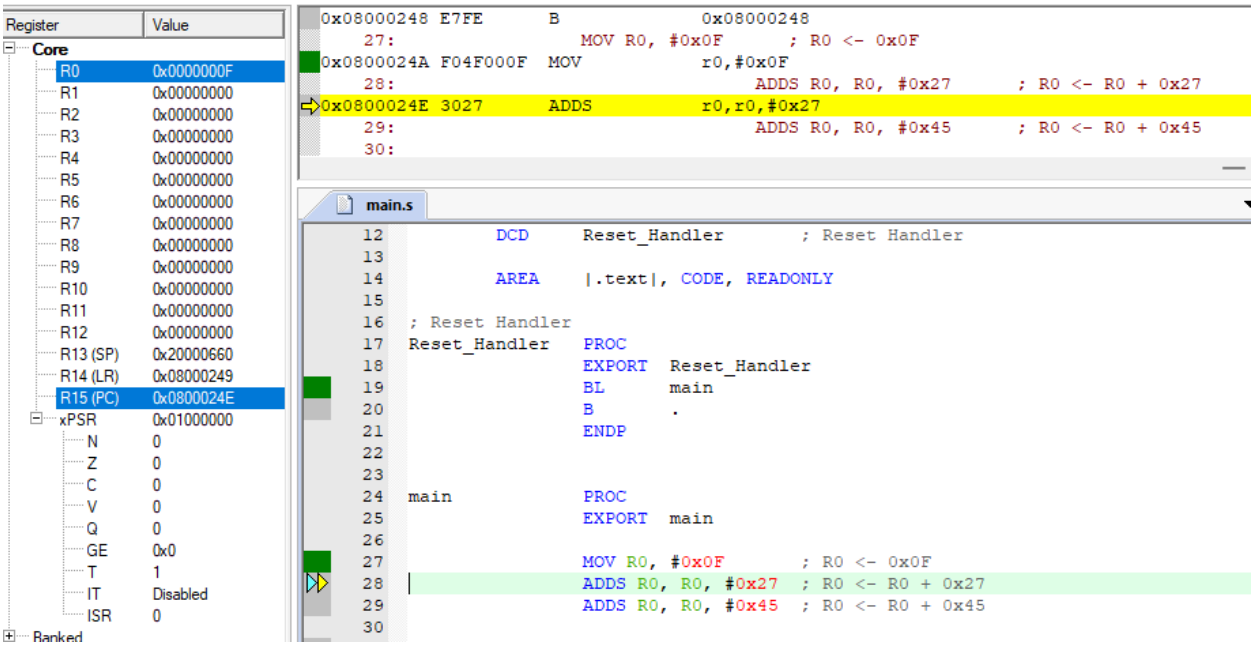
Task 5

RTL code:

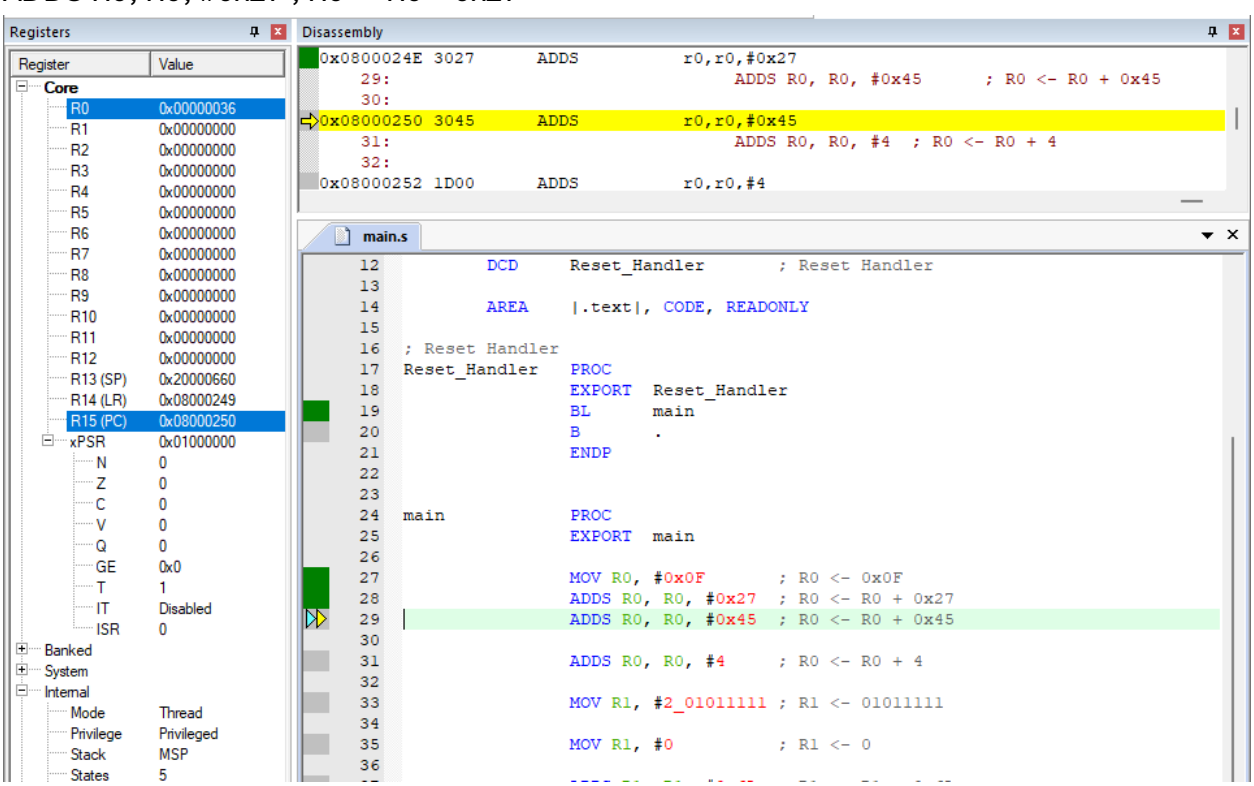
```
Reset_Handler   PROC
                EXPORT  Reset_Handler
                BL      main
                B       .
                ENDP


main            PROC
                EXPORT  main

                MOV R0, #0x0F       ; R0 <- 0x0F
                ADDS R0, R0, #0x27  ; R0 <- R0 + 0x27
                ADDS R0, R0, #0x45  ; R0 <- R0 + 0x45

                ADDS R0, R0, #4     ; R0 <- R0 + 4

                MOV R1, #2_01011111 ; R1 <- 01011111

                MOV R1, #0          ; R1 <- 0

                ADDS R1, R1, #0x6A  ; R1 <- R1 + 0x6A

                BX      LR              ; Return from main
                ENDP

                ALIGN
                END
```

Operations:

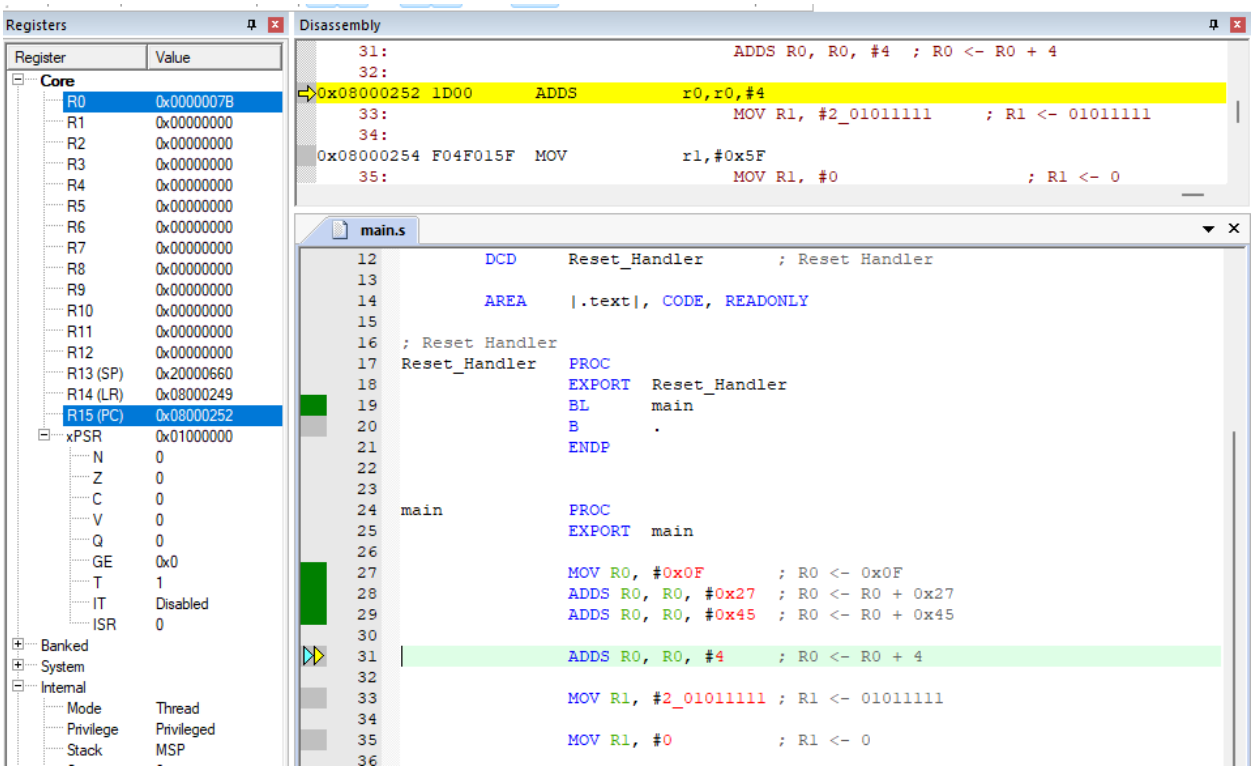
MOV R0, #0x0F ; R0 <- 0x0F



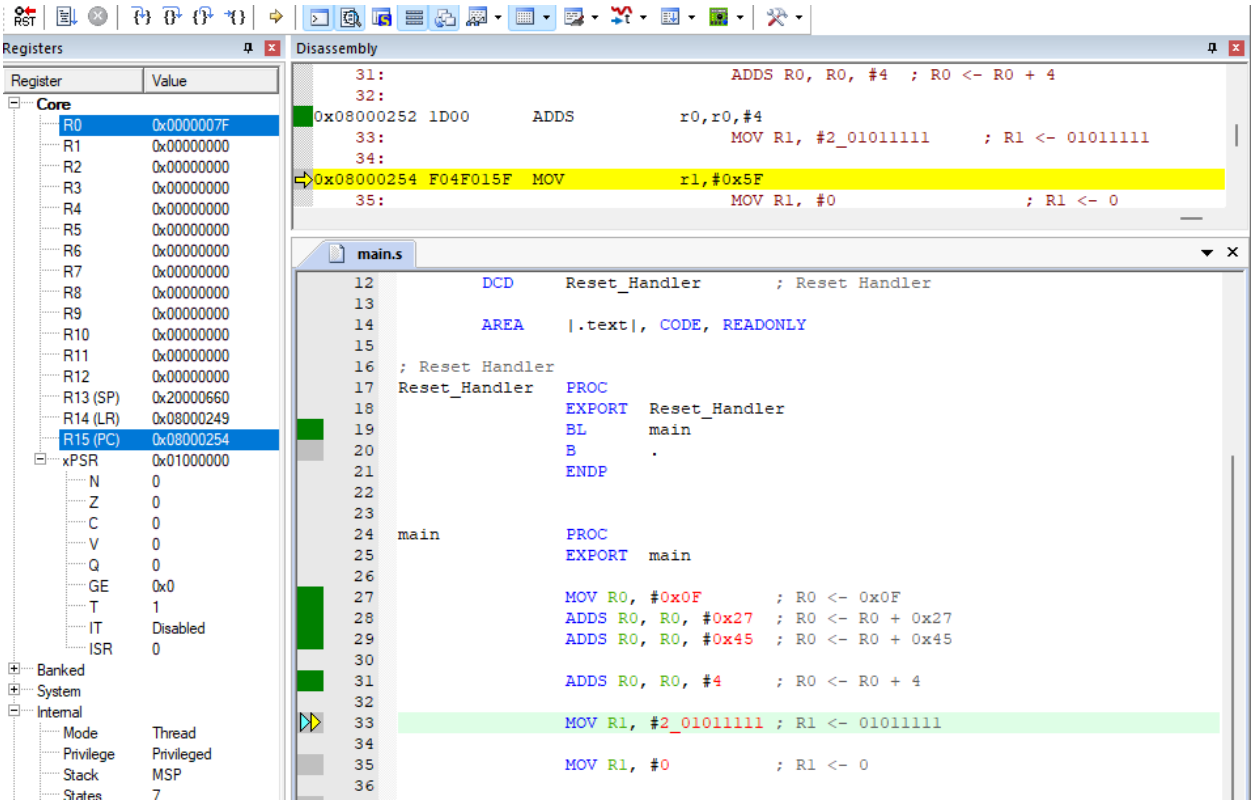
ADDS R0, R0, #0x27 ; R0 <- R0 + 0x27



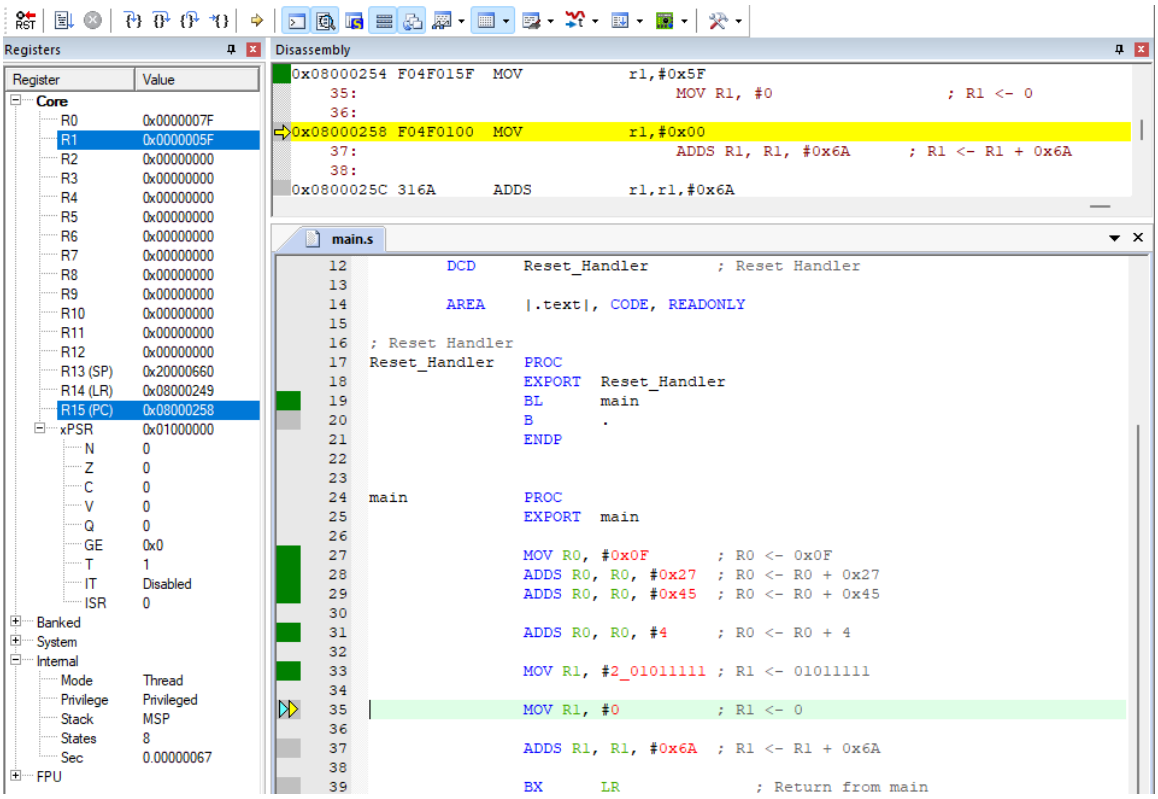
ADDS R0, R0, #0x45 ; R0 <- R0 + 0x45



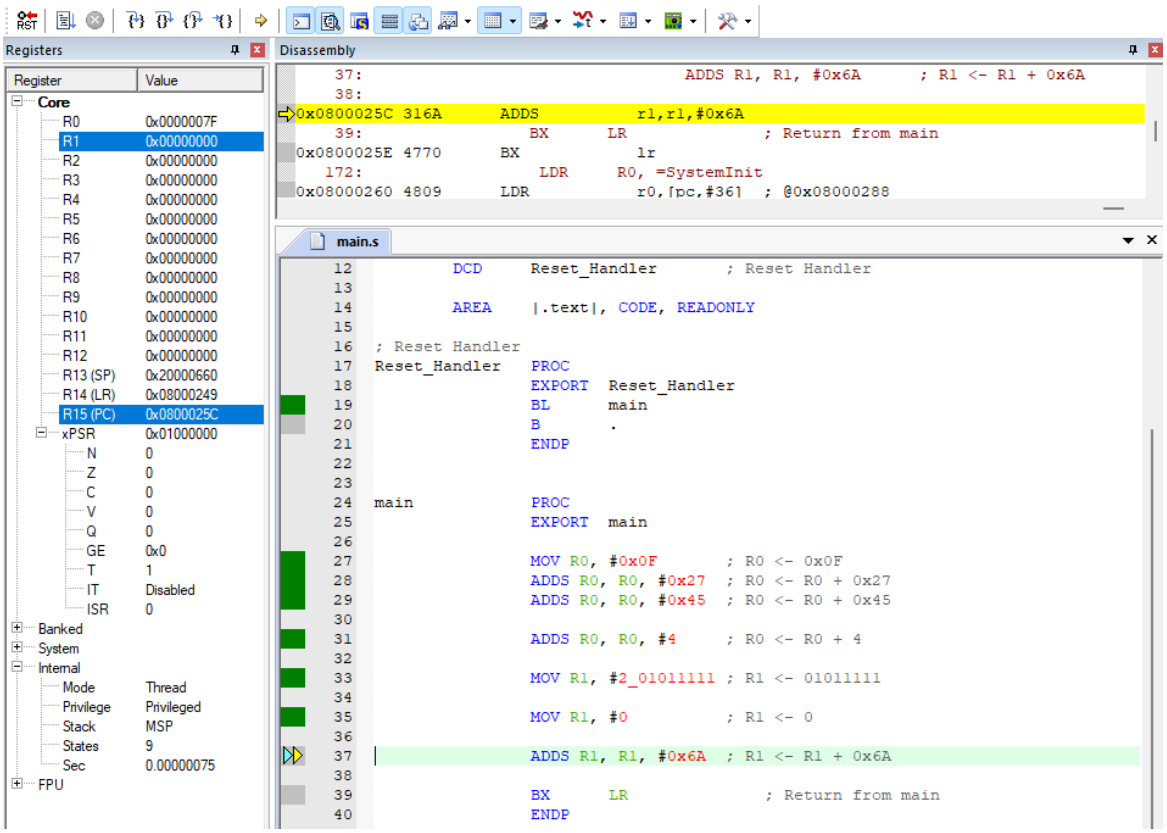
ADDS R0, R0, #4 ; R0 <- R0 + 4



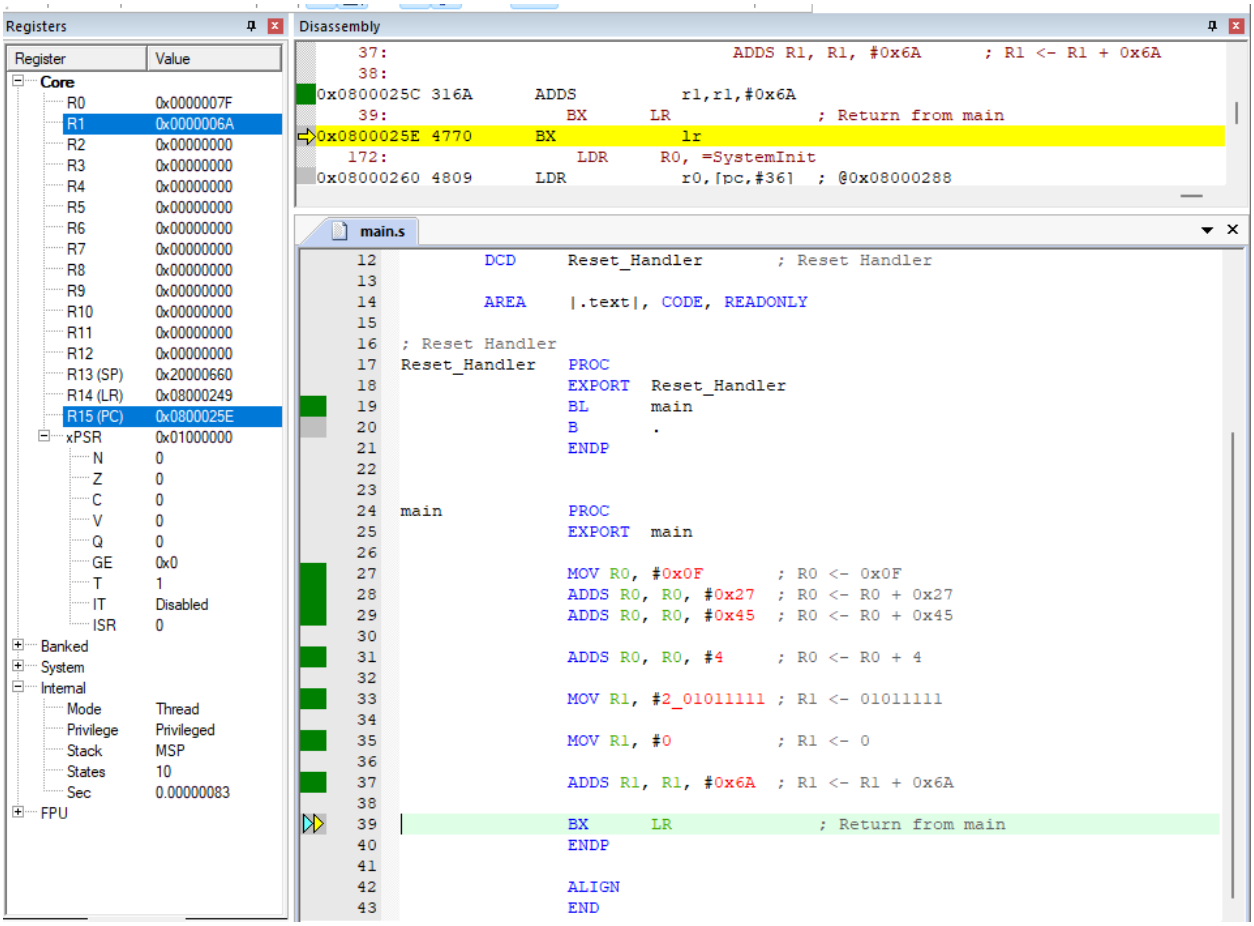
MOV R1, #2_01011111 ; R1 <- 01011111



MOV R1, #0 ; R1 <- 0



ADDS R1, R1, #0x6A ; R1 <- R1 + 0x6A


Registers
Register
Value
Core
R0 0x0000007F
R1 0x0000006A
R2 0x00000000
R3 0x00000000
R4 0x00000000
R5 0x00000000
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x00000000
R11 0x00000000
R12 0x00000000
R13 (SP) 0x20000660
R14 (LR) 0x08000249
R15 (PC) 0x0800025E
xPSR 0x01000000
N 0
Z 0
C 0
V 0
Q 0
GE 0x0
T 1
IT Disabled
ISR 0
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 10
Sec 0.00000083
FPU
Disassembly
37: ADDS R1, R1, #0x6A ; R1 <- R1 + 0x6A
38:
0x0800025C 316A ADDS r1,r1,#0x6A
39: BX LR ; Return from main
0x0800025E 4770 BX lr
172: LDR R0, =SystemInit
0x08000260 4809 LDR r0,[pc,#36] ; @0x08000288
main.s
12 DCD Reset_Handler ; Reset Handler
13
14 AREA |.text|, CODE, READONLY
15
16 ; Reset Handler
17 Reset_Handler PROC
18 EXPORT Reset_Handler
19 BL main
20 B .
21 ENDP
22
23
24 main PROC
25 EXPORT main
26
27 MOV R0, #0x0F ; R0 <- 0x0F
28 ADDS R0, R0, #0x27 ; R0 <- R0 + 0x27
29 ADDS R0, R0, #0x45 ; R0 <- R0 + 0x45
30
31 ADDS R0, R0, #4 ; R0 <- R0 + 4
32
33 MOV R1, #2_01011111 ; R1 <- 01011111
34
35 MOV R1, #0 ; R1 <- 0
36
37 ADDS R1, R1, #0x6A ; R1 <- R1 + 0x6A
38
39 BX LR ; Return from main
40 ENDP
41
42 ALIGN
43 END

Conclusion

Doing this lab I had more trouble with the hand calculations than anything. The keill uVision is good to use if the user has experience debugging.